

Universitatea POLITEHNICA din București  
Facultatea de Automatică și Calculatoare, Departamentul de  
Calculatoare



# LUCRARE DE DIPLOMĂ

## VisioScience3D

**Conducător Științific:**  
Moraru Anca Andreea

**Autor:**  
Dragomir Andrei-Mihai

București, 2025

University POLITEHNICA of Bucharest

Faculty of Automatic Control and Computers,  
Computer Science and Engineering Department



# BACHELOR THESIS

## VisioScience3D

**Scientific Adviser:**

Moraru Anca Andreea

**Author:**

Dragomir Andrei-Mihai

Bucharest, 2025

Realizarea acestui proiect nu ar fi fost posibilă fără ajutorul și sprijinul  
D-nei Prof.dr.ing Anca Andreea Moraru. Mulțumesc pentru îndrumare,  
promptitudine și răbdare.

Mulțumesc tuturor celorlalți profesori, laboratori și colegi care mi-au facilitat  
procesul de învățare și mi-au oferit suportul necesar de-a lungul acestor 4 ani, punându-mi  
bazele solide în domeniul ingineriei de Calculatoare.

# Abstract

Proiectul VisioScience3D vine ca un răspuns la nevoia de a crea un mediu de învățare interactiv și captivant pentru elevii din învățământul preuniversitar. Acesta îmbină tehnologia avansată pentru a crea un sistem de învățare bazat pe vizualizări 3D și simulări interactive, care să faciliteze înțelegerea conceptelor complexe din domeniul științelor exacte.

În momentul curent învățarea geometriei, fizicii, chimiei și altor discipline științifice se face prin metode tradiționale, care nu reușesc să capteze atenția elevilor. Prin acest proiect ne dorim să venim în întâmpinarea acestei nevoi, să oferim un mediu de învățare interactiv și captivant care să faciliteze activitatea didactică și să îmbunătățească rezultatele elevilor.

Proiectul VisioScience3D este o aplicație web care permite utilizatorilor să exploreze domeniul științelor exacte prin intermediul simulărilor interactive și vizualizărilor 3D. Acesta oferă o soluție inovatoare și complexă pentru elevi și profesori, având posibilitate să predea și să evalueze elevii prin intermediul platformei.

# Contents

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>Mulțumiri</b>                                           | <b>i</b>  |
| <b>Abstract</b>                                            | <b>ii</b> |
| <b>1 Introducere</b>                                       | <b>1</b>  |
| 1.1 Context                                                | 1         |
| 1.1.1 Defnirea problemei                                   | 1         |
| 1.1.2 Obiective                                            | 2         |
| 1.1.3 Susținere științifică                                | 2         |
| 1.2 Soluția propusă                                        | 3         |
| 1.3 Rezultate obținute                                     | 4         |
| 1.4 Structura lucrării                                     | 5         |
| <b>2 Analiza cerințelor / Motivația proiectului</b>        | <b>6</b>  |
| 2.1 Plaja de utilizatori                                   | 6         |
| 2.1.1 Categori                                             | 6         |
| 2.1.2 Profilul utilizatorului                              | 7         |
| 2.2 Motivația proiectului                                  | 8         |
| 2.3 Cerințe funcționale                                    | 8         |
| 2.3.1 Cerințe funcționale pentru utilizatorii elevi        | 8         |
| 2.3.2 Cerințe funcționale pentru utilizatorii profesori    | 9         |
| 2.3.3 Cerințe funcționale pentru sistem                    | 10        |
| 2.4 Cerințe nefuncționale                                  | 10        |
| 2.5 Limitări                                               | 12        |
| <b>3 Studiu de piață / Soluții existente</b>               | <b>14</b> |
| 3.1 Alte soluții existente                                 | 14        |
| 3.1.1 Colorado Edu                                         | 14        |
| 3.1.2 invatamate.com                                       | 15        |
| 3.1.3 Mozaweb.com                                          | 16        |
| 3.1.4 iqboard.ro                                           | 17        |
| 3.2 Raportarea la alte soluții                             | 18        |
| 3.3 Motivația alegerii VisioScience3D de către utilizatori | 18        |
| <b>4 Tehnologii utilizate pentru front-end</b>             | <b>20</b> |
| 4.1 Soluția de front-end                                   | 20        |
| 4.1.1 Framework principal                                  | 20        |
| 4.1.2 Framework 3D                                         | 22        |
| 4.1.3 Modelare 3D                                          | 23        |
| 4.2 Soluția UI/UX                                          | 24        |
| 4.2.1 Brandingul proiectului                               | 24        |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| 4.2.2    | Experiența utilizatorului . . . . .                       | 27        |
| 4.3      | Soluția de creare a scenelor 3D . . . . .                 | 29        |
| 4.3.1    | Utilizarea tehnologiilor WebGL și Three.js . . . . .      | 29        |
| 4.3.2    | Integrarea cu frontendul . . . . .                        | 31        |
| <b>5</b> | <b>Tehnologii utilizate pentru back-end</b>               | <b>32</b> |
| 5.1      | Descrierea arhitecturii . . . . .                         | 32        |
| 5.1.1    | Configurarea clusterului . . . . .                        | 32        |
| 5.1.2    | Structurarea și crearea fișierelor YAML . . . . .         | 33        |
| 5.1.3    | Gestionarea clusterului . . . . .                         | 36        |
| 5.1.4    | Rutarea și gestionarea traficului . . . . .               | 37        |
| 5.2      | Descrierea serviciilor . . . . .                          | 38        |
| 5.2.1    | Descrierea API-urilor . . . . .                           | 39        |
| 5.3      | Securitate . . . . .                                      | 40        |
| 5.4      | CI/CD . . . . .                                           | 40        |
| 5.4.1    | Port forwarding . . . . .                                 | 40        |
| 5.4.2    | Deployment . . . . .                                      | 40        |
| 5.4.3    | Metrici / Monitorizare . . . . .                          | 41        |
| 5.4.4    | Avantajele folosirii Prometheus . . . . .                 | 41        |
| 5.4.5    | Colectare Prometheus . . . . .                            | 41        |
| 5.4.6    | Grafana Dashboard . . . . .                               | 41        |
| 5.5      | Soluția de baze de date . . . . .                         | 44        |
| 5.5.1    | Structura bazei de date . . . . .                         | 44        |
| 5.5.2    | Securitatea datelor . . . . .                             | 45        |
| 5.5.3    | Integrarea cu back-endul . . . . .                        | 45        |
| <b>6</b> | <b>Detalii de implementare</b>                            | <b>46</b> |
| 6.1      | Back-end . . . . .                                        | 46        |
| 6.1.1    | Configurare back-end . . . . .                            | 47        |
| 6.1.2    | Dezvoltare back-end . . . . .                             | 48        |
| 6.1.3    | Conectivitate . . . . .                                   | 49        |
| 6.2      | Front-end . . . . .                                       | 51        |
| 6.2.1    | Configurare front-end . . . . .                           | 51        |
| 6.2.2    | Dezvoltare front-end . . . . .                            | 51        |
| 6.2.3    | Conectivitate . . . . .                                   | 53        |
| <b>7</b> | <b>Scenarii de utilizare</b>                              | <b>57</b> |
| 7.1      | Înregistrare și autentificare utilizator . . . . .        | 57        |
| 7.2      | Explorarea meniului principal . . . . .                   | 57        |
| 7.3      | Accesarea secțiunilor educaționale . . . . .              | 57        |
| 7.4      | Interacțiunea cu scenele 3D educaționale . . . . .        | 57        |
| 7.5      | Gestionarea profilului de profesor . . . . .              | 57        |
| 7.5.1    | Crearea de clase și gestionarea elevilor . . . . .        | 57        |
| 7.5.2    | Crearea de teste și gestionarea lor . . . . .             | 57        |
| 7.5.3    | Vizualizarea rezultatelor . . . . .                       | 57        |
| 7.6      | Gestionarea contului de elev . . . . .                    | 57        |
| 7.6.1    | Intrarea în clasele profesorului . . . . .                | 57        |
| 7.6.2    | Accesarea testelor și vizualizarea rezultatelor . . . . . | 57        |
| 7.6.3    | Rezolvarea testelor . . . . .                             | 57        |
| <b>8</b> | <b>Evaluarea implementării</b>                            | <b>58</b> |
| 8.1      | Evaluarea back-endului . . . . .                          | 58        |
| 8.1.1    | Testarea back-endului . . . . .                           | 58        |
| 8.1.2    | Monitorizarea back-endului . . . . .                      | 58        |

---

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| 8.1.3    | Evaluarea performanței back-endului . . . . .   | 58        |
| 8.2      | Evaluarea front-endului . . . . .               | 58        |
| 8.2.1    | Testarea front-endului . . . . .                | 58        |
| 8.2.2    | Monitorizarea front-endului . . . . .           | 58        |
| 8.2.3    | Evaluarea performanței front-endului . . . . .  | 58        |
| 8.3      | Testarea infrastructurii / Platformei . . . . . | 58        |
| <b>9</b> | <b>Concluzii și perspective</b>                 | <b>59</b> |
| 9.1      | Concluzii . . . . .                             | 59        |
| 9.2      | Dezvoltare viitoare . . . . .                   | 59        |
| <b>A</b> | <b>Anexe</b>                                    | <b>60</b> |

# Chapter 1

## Introducere

### 1.1 Context

Educația este un domeniu de bază al societății, iar tehnologia joacă un rol din ce în ce mai important în acest sector. În special, într-o lume în care platformele sociale și mediul de interacțiune video subsection de bază pentru tineri, este esențial să se dezvolte soluții educaționale care să fie atractive și eficiente în procesul clasic de învățare.

#### 1.1.1 Definirea problemei

În acest context, problema pe care o abordăm este crearea unei platforme educaționale interactive care să integreze tehnologii moderne și simulări 3D, pentru a face din procesul de învățare o experiență captivantă și eficientă pentru elevi de gimnaziu și liceu. Această platformă va permite accesul vizual și facil la informații complexe de matematică, fizică, chimie, astronomie și informatică.

Studentii vor putea explora concepte și interacționa cu simulări 3D, dar și să participe la teste și evaluări pentru a-și verifica cunoștințele. De asemenea, profesorii vor avea la dispoziție un instrument pentru a crea teste și a gestiona clasele de elevi, facilitând astfel procesul de predare și evaluare. Platforma va avea un sistem interactiv de navigare, recunoaștere a rezultatelor și răsplătirea progresului prin gamificare tot prin interacțiune 3D, ceea ce va îmbunătăți experiența utilizatorilor și va stimula învățarea activă.



### 1.1.2 Obiective

Obiectivele principale ale acestui proiect sunt:

- Crearea unei platforme educaționale interactive care să integreze simulări 3D și tehnologii moderne.
- Dezvoltarea unui sistem de gestionare a testelor și evaluărilor pentru profesori și elevi.
- Implementarea unui sistem de gamificare pentru a stimula învățarea activă și implicarea utilizatorilor.
- Asigurarea accesibilității și ușurinței în utilizare pentru elevi și profesori.
- Crearea unui mediu de învățare captivant și eficient care să faciliteze înțelegerea conceptelor complexe.
- Integrarea unui sistem de raportare și monitorizare a progresului utilizatorilor.
- Crearea unei interfețe prietenoase și intuitive care să faciliteze experiența profesorilor în gestionarea claselor, testelor, elevilor și a rezultatelor.

### 1.1.3 Susținere științifică

Multe discipline STEM implică concepte abstracte foarte dificil de vizualizat, ceea ce poate scădea interesul elevilor. De exemplu, chimia este adesea percepută ca “prea abstractă” deoarece elevii nu pot vizualiza ușor concepte precum structura moleculară sau reacțiile chimice.

Integrarea vizualizărilor 3D și a tehnologiilor interactive în predarea disciplinelor STEM este susținută de un număr semnificativ de cercetări recente. Acestea demonstrează că reprezentările vizuale și simulările contribuie la înțelegerea conceptelor abstracte și sporesc motivația elevilor.

De exemplu, un studiu derulat în școlile din Cehia a arătat că utilizarea modelelor 3D și animațiilor în predarea științelor a dus la o creștere semnificativă a implicării elevilor și a performanțelor la teste, în special în chimie și biologie [13]. De asemenea, o meta-analiză recentă a concluzionat că lecțiile care includ modele 3D interactive au îmbunătățit de peste 1,6 ori varianta standard de învățare teoretică [16].

Simulările 3D aplicate în laboratoare școlare au condus nu doar la o înțelegere mai bună a subiectelor, ci și la o retenție îmbunătățită a cunoștințelor în timp [17]. Elevii au raportat un nivel mai ridicat de încredere în propriile abilități și o atitudine mai pozitivă față de învățare.

Mai mult, numeroase cercetări evidențiază importanța predării adaptate stilurilor de învățare. Datele arată că un procent semnificativ dintre elevi învață predominant vizual, ceea ce justifică utilizarea elementelor grafice și a animațiilor în clasă [14], [15]. Un studiu local desfășurat în România confirmă această tendință, indicând o pondere de aproximativ 48% pentru stilul vizual, ceea ce subliniază necesitatea diversificării suportului educațional [14].

În plus, un raport OECD a demonstrat că utilizarea controlată a tehnologiei digitale în procesul educațional poate conduce la o creștere cu până la 15% a scorurilor obținute de elevi la testele de competențe, comparativ cu metodele clasice [19].

Ca și concluzie, dovezile sugerează că integrarea vizualizărilor 3D și a simulărilor interactive nu doar crește atractivitatea învățării, ci și eficiența ei. Proiectul *VisioScience3D* se aliniază acestor direcții moderne de predare, oferind resurse educaționale inovative care răspund nevoilor noilor generații de elevi.

## 1.2 Soluția propusă

Ideea platformei este de a crea un mediu de învățare interactiv care să integreze simulări 3D și tehnologii moderne pentru a face procesul de învățare mult mai ușor. Oferă o gamă de materii care pot fi studiate în această metodă inovativă, dar poate funcționa și ca verifcător ad-hoc al cunoștințelor elevilor. Un elev poate intra rapid și facil să verifice o formulă sau altă informație, iar profesorul poate să creeze teste și să gestioneze clasele de elevi în mod rapid și eficient.

Numele VisioScience3D a fost ales pentru a reflecta scopul platformei și este compus din două cuvinte: "Visio" care se referă la vizual, vedere, iar "Science" care se referă la știință. Această combinație sugerează o platformă care îmbină ideea de vizual cu știința, oferind un nume care reflectă esența platformei și scopul său. Titlul conține și termenul 3D, care subliniază focusul vizualizărilor din platformă care sunt realizate tridimensional.

Fiecare rol din procesul educațional (elev, profesor) are posibilitatea de accesa funcționalitățile de învățare și evaluare. De asemenea, platforma va avea un sistem de gamificare care va recompensa elevii pentru progresul lor și va încuraja participarea activă. Opțiuni de vizualizare a tabele de rezultate vor fi disponibile pentru profesori, iar elevii vor putea să-și urmărească scorul și progresul în timp real.

Testele pot fi create prin drag-and-drop în interfața secțiunii de create, unde există control

granular de la nivel de structura a quizului până la nivel de întrebare, răspuns, selecție de imagini sau număr de răspunsuri corecte. Profesorii pot vizualiza clasele pe care le dețin, elevii care au participat la teste și rezultatele obținute de aceștia. De asemenea, profesorii pot vizualiza și analiza rezultatele elevilor pentru a înțelege mai bine progresul acestora și pentru a adapta metodele de predare în funcție de nevoile fiecărui elev. Aceasta va permite o abordare personalizată a învățării, care poate îmbunătăți semnificativ rezultatele elevilor. Profesorii au acces și la sistemul de invitație a elevilor în platformă și în clasă direct în contul elevului.

Elevii pot accesa platforma printr-o interfață prietenoasă și intuitivă, unde pot explora concepte complexe prin simulări 3D și animații interactive. Pentru ei este destinat meniul 3D principal de selecție a materiei, unde pot vedea și interacționa cu toată gama de simulări disponibile. De asemenea, după cum am menționat mai sus, elevii pot participa la teste și evaluări pentru a-și verifica cunoștințele. Aceste teste sunt concepute pentru a fi interactive și captivante, oferind o experiență de învățare plăcută și eficientă, dar fiind și potrivite ad-hoc pentru o testare rapidă după o lecție predată.

### 1.3 Rezultate obținute

Platforma a ajuns într-un punct în care poate fi utilizată de către profesori și elevi pentru a explora concepte și reprezintă o soluție care poate salva timp și poate face învățarea mai rapidă și intuitivă pentru elevii cu stil de învățare vizual, care după cum am menționat și după cum arată studiile sunt majoritari în școlile din România (peste 48% din elevi). La nivelul de profesor reprezintă curent o soluție rapidă de testare știința monitorizare a elevilor la materii de știință, dar și de informatică.

La nivel tehnic, platforma folosește o arhitectură scalabilă a serviciilor din back-end, oferind o scalabilitate foarte ridicată și o disponibilitate crescută datorită separării în microservicii a aplicației. Arhitectura poate fi ușor extinsă pentru a adăuga noi funcționalități și module, iar platforma poate fi adaptată rapid la nevoile utilizatorilor. De asemenea, partea de front-end este construită folosind tehnologii cu suport extins și comunități mari, ceea ce asigură o suport îndelungat și o posibilă dezvoltare ușoară a platformei în viitor.

## 1.4 Structura lucrării

Această lucrare este structurată în mai multe capitole, fiecare abordând un aspect diferit al proiectului atât din punct de vedere tehnic, cât și din punct de vedere al implementării și utilizării platformei.

Capitolul 2 oferă o analiză detaliată a cerințelor și a nevoilor utilizatorilor, precum și a profilului utilizatorilor tipici ai platformei. Acesta include o descriere a motivației, a cerințelor funcționale și a celor nefuncționale, precum și a limitărilor și constrângerilor proiectului.

Al treilea capitol (3) analizează piața și competiția din punct de vedere al platformelor educaționale existente, evidențiind punctele forte și slabe ale acestora și modul în care platforma propusă se diferențiază de cea dezvoltată în cadrul acestui proiect.

Capitolul 4 detaliază tehnologiile utilizate în dezvoltarea platformei, inclusiv limbajele de programare, framework-urile și instrumentele utilizate pentru a crea aplicația. Aici se oferă o privire de ansamblu asupra dezvoltării fiecărei părți importante a aplicației: back-end, front-end (inclusiv UI/UX), scene 3D și bazele de date.

Capitolul 6 oferă detalii despre implementarea platformei, inclusiv bucăți de cod și exemple de dezvoltare a codului, precum și detalii despre configurarea și conectivitatea obținută între diferitele componente ale aplicației.

Cel de-al șaselea capitol, 7 are ca focus definirea și prezentarea scenariilor de utilizare ale platformei, inclusiv modul în care utilizatorii pot interacționa cu aplicația și cum pot crea conturi, teste, clase și cum pot vizualiza scene și experimente 3D. De asemenea, se discută despre modul în care utilizatorii pot accesa și utiliza funcționalitățile platformei, precum și despre modul în care pot beneficia de gamificare și recompense pentru progresul lor.

Capitolul 8 se concentrează pe tipurile de evaluare a platformei, cu accent pe testare și pe modul în care se poate monitoriza sănătatea platformei în cazul lansării către publicul larg. De asemenea se discută și despre evaluare performanței platformei pe diferite niveluri.

Ultimul capitol, 9, oferă concluzii și perspective asupra viitorului platformei, inclusiv posibile îmbunătățiri și extinderi ale funcționalităților existente. De asemenea, se discută despre impactul pe care platforma ar putea să-l aibă asupra educației.

## Chapter 2

# Analiza cerințelor / Motivația proiectului

### 2.1 Plaja de utilizatori

Publicul țintă al aplicației este destul de general, putând fi accesată de orice utilizator care se înregistrează și dorește să învețe sau să își amintească noțiuni de matematică, fizică, chimie, astronomie sau informatică.

#### 2.1.1 Categorii

Categoriile principale de utilizatori suportate de platformă sunt:

- Utilizatori elevi
- Utilizatori profesori

Aceste categorii indică și publicul țintă al aplicației, care este format din elevi de nivel gimnaziu sau liceu și profesori care predau disciplinele menționate la acest nivel de învățământ.

Între aceste două categorii de utilizatori diferă drepturile de acces dar și tipul de interacțiune cu aplicația dar și funcționalitățile disponibile.

### 2.1.2 Profilul utilizatorului

După cum am menționat anterior, aplicația este destinată elevilor și profesorilor de matematică, fizică, chimie, astronomie și informatică. Putem face și o analiză din diverse puncte de vedere demografice și comportamentale:

- **Vârstă:** 10-18 ani pentru elevi, 25-60 ani pentru profesori

Motivat de faptul că aplicația este destinată elevilor de gimnaziu și liceu.

- **Tip de învățare:** vizual

Justificat de faptul că aplicația folosește animații 3D pentru a explica concepte științifice.

- **Studii:** elevi de gimnaziu și liceu, profesori cu studii superioare în domeniul educației sau al științelor exacte

Motivat de scopul și ținta aplicației.

- **Locație:** România, limba destinată fiind româna

Justificat de faptul că aplicația este destinată elevilor și profesorilor din România.

- **Interese:** educație, tehnologie, știință, învățare interactivă

Justificat de faptul că are suport pentru materii de știință și tehnologie.

- **Motivație elevi:** dorința de a învăța și de a-și îmbunătăți cunoștințele în domeniile menționate, dorința de a obține note mai mari la școală

Motivat de faptul că aplicația este destinată elevilor care doresc să învețe și să-și îmbunătățească cunoștințele.

- **Motivație profesori:** dorința de a-și îmbunătăți metodele de predare, dorința de a oferi elevilor o experiență de învățare mai interactivă și mai captivantă

Justificat de faptul că aplicația este destinată profesorilor care doresc să-și îmbunătățească metodele de predare.

- **Tehnologie:** utilizatori cu un nivel cel puțin începător-mediu de competență tehnologică, familiarizați cu utilizarea aplicațiilor web

Datorită faptului că aplicația este o aplicație web care necesită cunoștințe de bază în accesare și navigarea pe internet.

- **Dispozitive:** utilizatori care folosesc computere, laptopuri, tablete sau telefoane mobile pentru accesarea aplicației

Motivat de faptul că aplicația este o aplicație web care poate fi accesată de pe orice dispozitiv cu acces la internet.

## 2.2 Motivația proiectului

Motivația proiectului este de a oferi o platformă educațională interactivă care să ajute elevii, decizia ideii fiind luată după ce s-a studiat piața de soluții existente care oferă acest tip de produs destinat României, în limba română și care oferă o experiență modernă, cât și suport pentru testare încorporat.

## 2.3 Cerințe funcționale

interfața a fost realizată după două principii de bază:

- **Simplitate:** am ales un design simplu, minimalist, care să nu distragă atenția utilizatorului de la conținutul educațional
- **Interactivitate:** am ales să folosim animații 3D în cât mai multe zone ale aplicației, pentru a face experiența de învățare mai captivantă și mai plăcută
- **Accesibilitate:** am ales să folosim o paletă de culori care să fie ușor de citit și să nu obosească ochii utilizatorului

Cerințele funcționale sunt împărțite în două mari categorii, în funcție de tipul de utilizator:

- Cerințe funcționale pentru utilizatorii elevi
- Cerințe funcționale pentru utilizatorii profesori
- Cerințe funcționale pentru sistem

### 2.3.1 Cerințe funcționale pentru utilizatorii elevi

Cerințele funcționale pentru utilizatorii elevi sunt cerințe care țin de utilizatorii aplicației și modul în care aceștia pot să interacționeze cu aplicația din poziția de elevi.

Printre cele principale se numără:

- Utilizatorii elevi trebuie să se poată înregistra în aplicație

- Utilizatorii elevi trebuie să se poată autentifica în aplicație
- Utilizatorii elevi trebuie să aibă acces la un meniu principal 3D de selectare a materiei
- Utilizatorii elevi trebuie să aibă acces la secțiuni educaționale
- Utilizatorii elevi trebuie să aibă acces la scenele 3D și să poată interacționa cu ele
- Utilizatorii elevi trebuie să poată intra în clase create de profesori prin invitație
- Utilizatorii elevi trebuie să poată răspundă la invitații
- Utilizatorii elevi trebuie să aibă acces să vizualizeze testele deschise
- Utilizatorii elevi trebuie să aibă să rezolve testele deschise
- Utilizatorii elevi trebuie să aibă acces la rezultatele obținute dacă profesorul a ales să le facă publice
- Utilizatorii elevi trebuie să aibă acces profilul de utilizator de gestiune a contului

### 2.3.2 Cerințe funcționale pentru utilizatorii profesori

Cerințele funcționale pentru utilizatorii profesori sunt cerințe care țin de utilizatorii din categoria profesorală și care sunt necesare pentru a asigura o experiență adecvată lor.

Cele mai importante sunt:

- Utilizatorii profesori trebuie să se poată înregistra în aplicație
- Utilizatorii profesori trebuie să se poată autentifica în aplicație
- Utilizatorii profesori trebuie să aibă acces la un meniu principal 3D de selectare a materiei
- Utilizatorii profesori trebuie să aibă acces la secțiuni educaționale
- Utilizatorii profesori trebuie să aibă acces la scenele 3D și să poată interacționa cu ele
- Utilizatorii profesori trebuie să aibă acces la un meniu de gestionare a elevilor și claselor
- Utilizatorii profesori trebuie să aibă acces la un meniu de gestionare a testelor
- Utilizatorii profesori trebuie să aibă acces la un meniu de vizualizare a rezultatelor obținute de elevi
- Utilizatorii profesori trebuie să aibă acces profilul de utilizator de gestiune a contului
- Utilizatorii profesori trebuie să aibă acces la un meniu de gestionare a invitațiilor



- Utilizatorii profesori trebuie să aibă acces să genereze invitații pentru elevi
- Utilizatorii profesori trebuie să aibă acces să trimită invitații pentru elevi
- Utilizatorii profesori trebuie să poată vedea și gestiona rezultatele elevilor
- Utilizatorii profesori trebuie să poată crea teste și închide/deschide accesul la ele

### **2.3.3 Cerințe funcționale pentru sistem**

Cerințele funcționale pentru sistem sunt cerințe care nu țin de utilizatorii aplicației, ci de sistemul în sine și modul în care acesta trebuie să funcționeze. Aceste cerințe sunt esențiale pentru a asigura o experiență de utilizare fluidă și eficientă.

- Sistemul trebuie să trimită invitațiile aproape instantaneu către utilizatori
- Sistemul trebuie să trimită notificări utilizatorilor atunci când primesc invitații
- Sistemul trebuie să încarce rapid scenele 3D
- Sistemul trebuie să aibă un timp de răspuns rapid la interacțiunile utilizatorilor
- Sistemul trebuie să răspundă instant la interacțiunile utilizatorilor cu scenele 3D
- Sistemul trebuie să ofere rezultatele la teste instant
- Sistemul trebuie să gestioneze eficient sistemul de utilizatori și autentificare
- Sistemul trebuie să gestioneze sistemul de închidere/deschidere a testelor

## **2.4 Cerințe nefuncționale**

Dezvoltarea produsului a fost destinată pentru utilizare pe Web, de pe orice dispozitiv cu acces la internet, fără a necesita instalarea de aplicații sau plugin-uri suplimentare. Din natura aplicației, gradul de portabilitate de la dispozitiv la dispozitiv este ridicat.

Interfața trebuie să răspundă rapid la interacțiunile utilizatorilor, iar animațiile 3D să fie fluide și să nu aibă întârzieri semnificative. De asemenea, aplicația trebuie să fie disponibilă pe toate browserele moderne, inclusiv Chrome, Firefox, Safari și Edge.

Pentru o utilizare mai intuitivă sunt adăugate în unele locuri și indicatoari vizuali care arată modul de funcționare. Putem observa și un exemplu în figura următoare, unde utilizatorul este îndrumat să pună mouse-ul peste un obiect pentru a activa meniul.

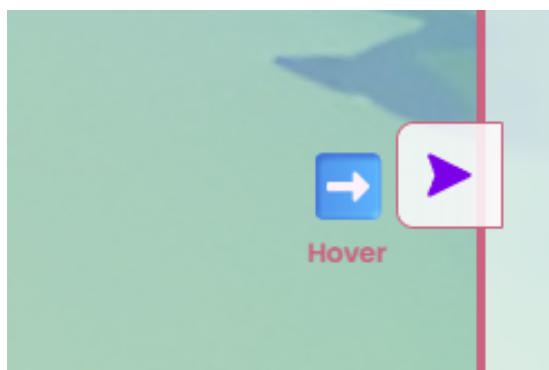


Figure 2.1: Animație intuitivă pentru utilizator deschiderea unui meniu lateral

Meniurile conțin informații de prezentare și tutoriale pentru a naviga diferitele simulări 3D, ele fiind primul lucru pe care utilizatorul îl vede la intrarea în diferitele secțiuni. Putem vedea un exemplu în figura următoare.

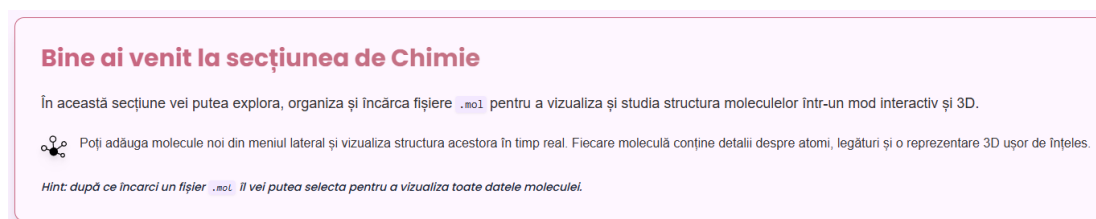


Figure 2.2: Meniu de întâmpinare și ghidare pentru utilizator

Datele afișate în platformă sunt actualizate constant prin comunicare cu serverul care gestionează acel serviciu prin protocolul HTTP.

Ca cerințe nefuncționale necesare pentru încărcarea și gestionarea corectă a scenelor 3D, putem să discutăm pe mai multe planuri, cum ar fi:

## 1. Browser & API

Aplicația trebuie să ruleze pe browsere moderne care suportă WebGL 1.0 (OpenGL ES 2.0) sau, de preferat, WebGL 2.0 (OpenGL ES 3.0). Browsere compatibile: Chrome > 60, Firefox > 52, Safari > 11, Edge > 16.

## 2. CPU

- **Minim desktop:** Dual-core 1,5 GHz (Intel Core i3 gen.3+, AMD A6+)
- **Minim mobil/tablet:** Dual-core ARM Cortex-A55 @1,8 GHz

### 3. Memorie (RAM)

- **Minim:** 2 GB RAM total, din care  $\geq 256$  MB cel puțin liberi pentru heapul JavaScript și texturi.
- **Recomandat:** 4 GB+ RAM.

### 4. GPU & VRAM

- **Minim placă grafică integrată:** Intel HD 5000 / AMD Radeon R5 ( $> 128$  MB VRAM)
- **Minim placă grafică dedicată:** NVIDIA GT 640 / AMD R7 250 ( $> 256$  MB VRAM)
- **Minim mobil/tablet:** Adreno 306 / Mali-T720 ( $> 128$  MB VRAM)

### 5. Rețea

- **Lățime de bandă minimă:** 1 Mbps (cu asset-uri comprimate și LOD).
- **Latency:**  $< 100$  ms RTT pentru încărcarea resurselor.

### 6. Performanță în aplicație

- **FPS țintă:**  $> 30$  fps pe desktop,  $> 24$  fps pe mobil.
- **Timp de încărcare inițială:**  $< 2$  s,  $< 5$  s (la texturile mari).

Aceste cerințe au fost testate în mediul de dezvoltare și sunt valabile pentru majoritatea, putând să difere în mediul de producție în cazul lansării aplicației.

## 2.5 Limitări

Din punct de vedere al specificului platformei, ea oferă limitări clare pe mai multe planuri, cum ar fi:

- **Compatibilitate browser și platformă:** aplicația este compatibilă cu majoritatea browserelor moderne, dar nu este optimizată pentru browserele mai vechi sau pentru dispozitive mobile cu specificații tehnice reduse.
- **Performanță hardware:** aplicația necesită un hardware minimal pentru a funcționa corect.

- **Conexiune la internet:** aplicația necesită o conexiune la internet stabilă pentru a funcționa corect.
- **Dispozitive mobile și tabletă:** aplicația necesită un hardware decent pentru rulare, ceea ce e mai greu de obținut pentru dispozitivele mobile.
- **Practice:** aplicația are limitări în folosire încât este dependentă în cea ce constă eficacitatea învățării de utilizatorul care o folosește.
- **Conținut:** aplicația nu oferă un conținut educațional complet, ci doar partea modelabilă 3D a acestuia per fiecare materie.

## Chapter 3

# Studiu de piață / Soluții existente

### 3.1 Alte soluții existente

#### 3.1.1 Colorado Edu

Prima soluție analizată este Colorado Edu, o platformă care oferă acoperire pe multiple materii după cum se poate observa în captura următoare.

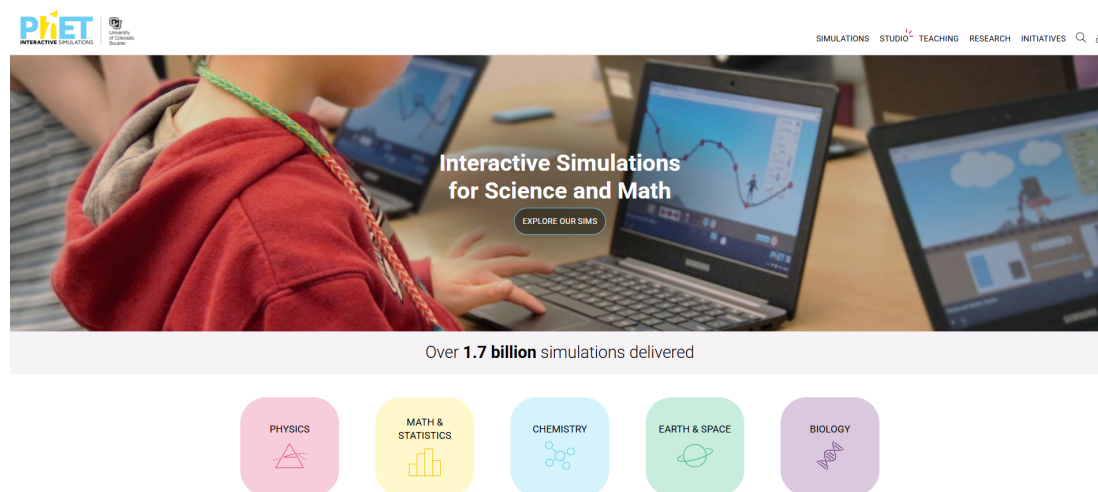


Figure 3.1: Captură de ecran de pe soluția Phet Colorado Edu

Se poate observa designul simplu și intuitiv și suportul relativ extins pe materii. Are și o interactivitate bună la nivel de simulări și alt avantaj e că are acces gratuit. Totuși, nu are un sistem de evaluare integrat și nu permite crearea de teste personalizate. De asemenea, nu are un sistem de management al utilizatorilor, ceea ce face ca utilizarea platformei să fie limitată

la simulări ad-hoc, iar ținta principală ca științelor curriculum este SUA și nu România.

### 3.1.2 invatamate.com



Figure 3.2: Captură de ecran de pe platforma invatamate.com

Cea de-a doua soluție analizată este invatamate.com, o platformă care oferă o suită de cursuri, vizualizări și simulări online. După cum se poate observa în captura de ecran, platforma are un design relativ învechit și nu foarte atractiv, dar conține multiple resurse utile. Deși numele sugerează că platforma este dedicată matematicii, ea are și simulări de astronomie și jocuțe interactive mai generale.

Problema este iar că nu are un sistem de management al utilizatorilor și nici suport pentru evaluare integrat. Lipsa acestor funcționalități face ca platforma să nu fie foarte utilă în mediul educațional, iar utilizarea ei să fie limitată la jocuțe simple și triviale. Alte limitări ale platformei sunt că folosește Flash pentru majoritate jocurilor științelor chiar integrări cu Kahoot pentru unele, iar resursele sunt de asemenea legate din surse externe în unele cazuri. De asemenea, experiența nu este cea mai modernă, intuitivă și plăcută după cum se poate vedea în capturile următoare.

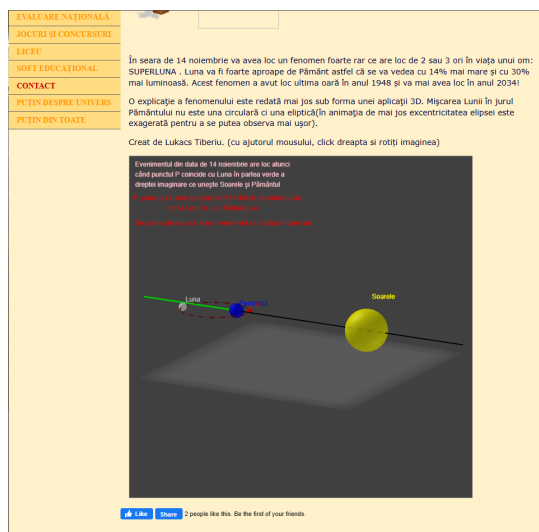


Figure 3.3: Simulări de pe platforma invatamate.com

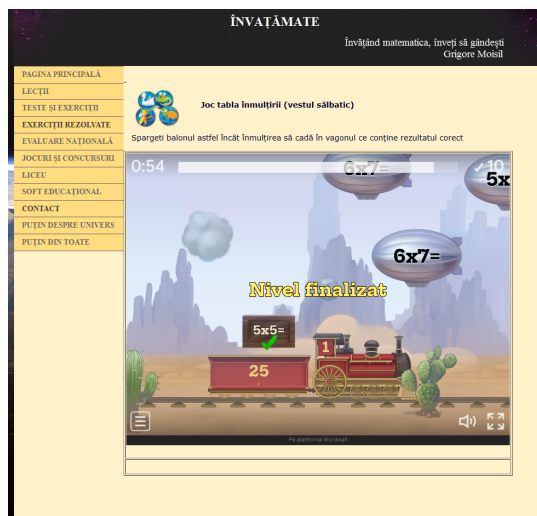


Figure 3.4: Jocuri de pe platforma invatamate.com

### 3.1.3 Mozaweb.com

Această soluție are o arhitectură cu produse pentru mai multe roluri (pentru elevi, pentru profesori, pentru școli) și o experiență de utilizare plăcută și intuitivă. Se poate observa în captura de ecran de mai jos cum arată interfața de întâmpinare a utilizatorului.

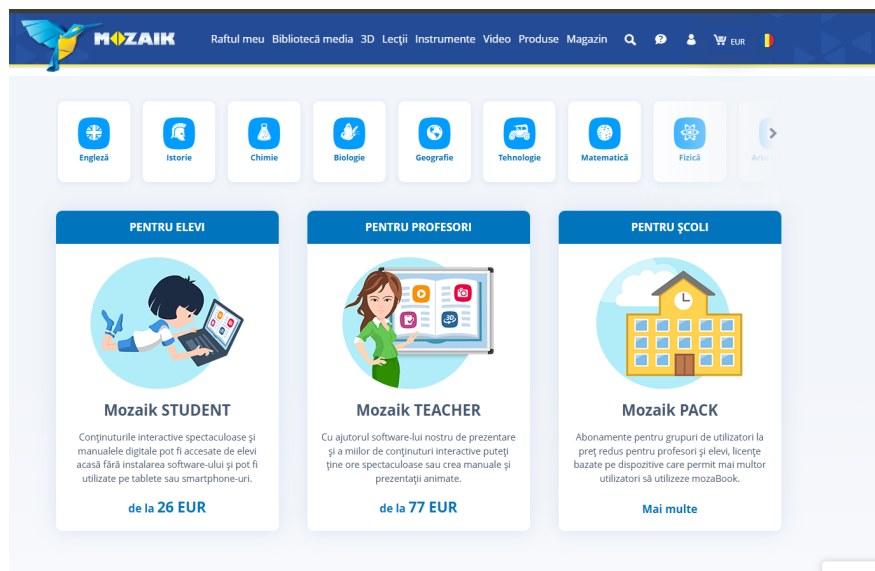


Figure 3.5: Captură de ecran de pe platforma mozaweb.com

Se poate observa suportul pentru numeroase materii și din zona STEM, dar și pentru alte domenii. De asemenea, platforma are simulări foarte profesioniste și interactive, dar dezavan-

tajele mari sunt prețurile abonamentelor pentru că discutăm de o platformă comercială și nu gratuită. De asemenea pentru simulări este necesar să se instaleze un plugin extern, reducând astfel accesibilitatea platformei. Se poate observa mai jos gama de simulări și necesitatea plugin-ului.

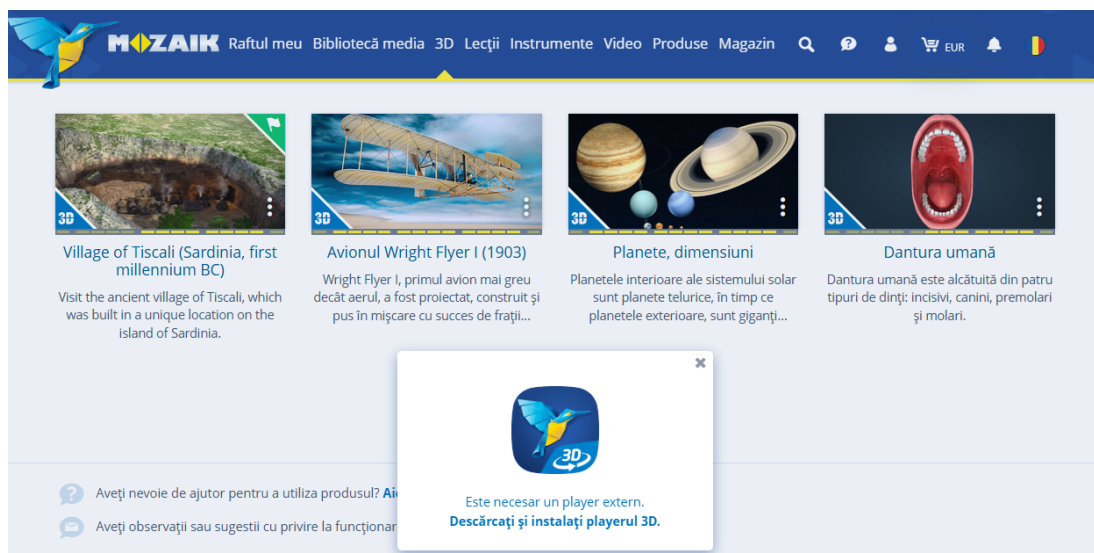


Figure 3.6: Simulări și dovada plugin-ului necesar pentru a le rula

Ținta aceste platforme este una comercială mai mult decât educațională, iar prețurile sunt relativ mari pentru toate tipurile de utilizatori.

### 3.1.4 iqboard.ro

Ultima soluție analizată este iqboard.ro, o platformă care oferă o suită de resurse educaționale în limba română, cu multiple moduri de lucru, multi touch, bazat pe un sistem de abonamente. Are o interfață interesantă și atractivă și suport extins educațional după cum se poate observa în captura de ecran de mai jos (figura 3.7).

Dezavantajele majore sunt prețurile extrem de mari pentru utilizatori (la nivel de sute de euro după cum se poate observa în figura 3.7). Prețul este relativ nejustificat deși materialele sunt de calitate bună și platforma are un design interactiv și atractiv. În figura 3.8 se poate urmări un exemplu de simulare disponibilă pe platformă. Platforma are moduri separate pentru pregătire, predare și mod desktop, dar nu are un sistem de management al utilizatorilor sau vreun sistem de evaluare sau urmărire a progresului utilizatorilor.

Ca alte funcționalități, platforma se laudă cu modul multi-user care poate fi folosit în modul tabla sau full-screen. După selectarea instrumentului dorit, utilizatorii pot lucra simultan pe



tabla, cu modul multi-screens (modul petrecere), cu posibilitate de folosire simultana a doua table interactive (modul MX2) și cu modul raspuns interactiv.

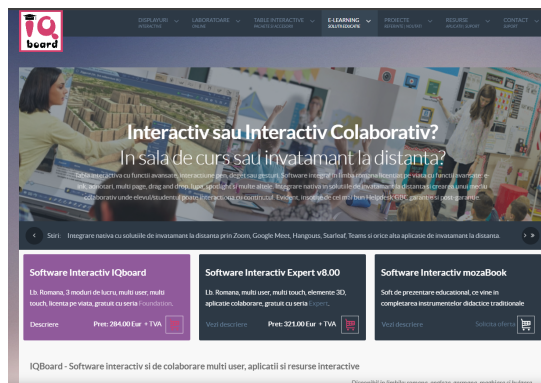


Figure 3.7: Captură de ecran de pe platforma iqboard.ro



Figure 3.8: Simulări de pe platforma iqboard.ro

### 3.2 Raportarea la alte solutii

- **O singură platformă completă pentru toate materiile STEM** În timp ce Phet sau Invatamate se concentrează mai mult pe un subiect, VisioScience3D va oferi un mediu unificat cu module interactive de matematică, fizică, chimie, informatică și chiar astronomie, reducând necesitatea schimbării constante de aplicații.
- **Simulări 3D real-time și configurabile** Mozaweb de exemplu are mult suport pentru scene 3D, dar ele sunt predefinite și nu pot fi adaptate dinamic. VisioScience3D permite ajustarea parametrilor în timp real (forțe, unghiuri, constante) și animarea vectorilor.
- **Dashboard de monitorizare a progresului** Lipsa de raportare în timp real este o problemă la majoritatea soluțiilor gratuite. Profesorii pot vizualiza instantaneu statistici legate de scoruri, dificultăți întâmpinate și pot adapta testele în funcție de nevoile elevilor.
- **Gamificare** Majoritatea platformelor nu au un sistem de gamificare bine definit. VisioScience3D va include elemente de gamificare pentru a spori implicarea elevilor.

### 3.3 Motivatia alegerii VisioScience3D de către utilizatori

VisioScience3D răspunde foarte bine nevoilor profesorilor și elevilor de azi. Oferă simultan simulări 3D real-time pentru toate disciplinele STEM, configurabile direct în browser în platformă, fără pluginuri externe. În timp ce multe platforme existente fie acoperă doar un număr

limitat de subiecte, fie implică licențe costisitoare sau dependențe externe (Mozaweb, IQBoard). VisioScience3D pune la dispoziție un mediu unic, adaptat programei românești. Profesorii pot monitoriza progresul elevilor prin dashboard-uri integrate și primesc rapoarte detaliate. Prin modelul gratis și nivelul simulărilor suportate, VisioScience3D este un ecosistem complet, flexibil și orientat spre rezultatele elevilor.

## Chapter 4

# Tehnologii utilizate pentru front-end

### 4.1 Soluția de front-end

#### 4.1.1 Framework principal

Frameworkul principal pe care l-am folosit este React JS, care este o tehnologie foarte populară și răspândită bazată pe JavaScript. În alegerii de tehnologie pentru front-endul platformei VisioScience3D, am concentrat atenția pe criterii precum maturitatea ecosistemului, performanță, curba de învățare, flexibilitate în personalizare și capacitatea de integrare cu biblioteci 3D. Dintre principalele opțiuni (Vue.js, Angular, Svelte), React s-a impus datorită următoarelor avantaje:

- **Ecosistem matur și susținere largă:** React are o comunitate activă de milioane de dezvoltatori, cu un număr impresionant de librării, tutoriale și un suport consistent din partea Meta. Această resursă ne-a permis să adoptăm rapid bune practici și să găsim soluții la provocări specifice dezvoltării 3D.
- **Model declarativ și component-driven:** React oferă un mod de declarare detaliată în descrierea interfeței, ideal pentru scene 3D complexe, unde starea se propagă predictibil prin componente. Aceasta ușurează managementul ciclului de viață al obiectelor din biblioteca 3D, reducând riscul de actualizări inconsistente.

- **React Router DOM:** Pentru că oferă o soluție de rutare foarte ușoară în navigarea între diferitele secțiuni ale platformei (fizică, chimie, astronomie etc.) prin React Router DOM ce oferă simplitate în definirea rutelor și suportul pentru încărcare întârziată de module. Alternativele (Vue Router, Angular Router) sunt la fel de capabile, însă integrarea lor cu React Three Fiber, care vom vedea este baza în simulările 3D, ar fi impus un strat suplimentar de interoperabilitate.

Pe zona de stilizare a interfeței, am optat pentru Tailwind CSS, un framework CSS utilitar care ne-a permis să creăm un design personalizat. Am comparat framework-uri clasice de CSS (Bootstrap, Material UI) cu Tailwind CSS și am observat următoarele avantaje:

- **Utilitare-întâi și JIT:** Tailwind oferă clase utilitare care permit definirea rapidă a layout-urilor și a stilurilor unice, fără a scrie sutele de linii de CSS personalizat. Motorul JIT (Just-In-Time) generează doar clasele folosite, menținând pachetul final mai mic.
- **Consistență și flexibilitate:** În loc să ne încărcăm aplicația cu componente predefinite, am putut construi una interfață coerentă, respectând paleta de culori și spațiile dorite, fără a rescrie componente UI complexe.
- **Integrare strânsă cu React:** Utilizarea `className` din JSX se potrivește natural cu Tailwind, iar plugin-urile precum `@tailwindcss/forms` facilitează stilizarea formulelor și input-urilor.
- **Extensibilitate:** Tailwind permite personalizarea ușoară a temelor, adăugarea de plugin-uri și crearea de componente reutilizabile, fără a sacrifica viteza de dezvoltare.

Bibliotecile de creare și gestionare a graficii 3D sunt esențiale pentru platforma noastră, iar pentru a crea simulări interactive, am ales Three.js, o bibliotecă JavaScript populară. Three.js oferă un API puternic pentru crearea de scene 3D, animații și interacțiuni și Împreună cu React Three Fiber (R3F), o bibliotecă care integrează Three.js cu React, am putut să ne concentrăm pe dezvoltarea rapidă a aplicațiilor 3D fără a ne preocupa de detaliile de implementare ale Three.js. Utilizarea peste a Drei, ce conține o suită de componente și utilitare pentru R3F, a permis accelerarea procesului de dezvoltare, oferind soluții gata făcute pentru anumite probleme comune.

Câteva dintre beneficiile integrării cu Three.js prin React Three Fiber și Drei care au făcut alegerea justificată sunt:

- **Abstracție declarativă:** R3F “împachetează” API-ul imperativ Three.js într-un DSL React, permițându-ne să definim scene, obiecte și materiale în JSX, sub formă de componente. Astfel, putem folosi hook-uri (`useFrame`, `useLoader`) pentru a sincroniza animațiile și resursele fără prea mult cod duplicat.
- **Ecosistem Drei:** Biblioteca Drei aduce peste 100 de componente existente (`OrbitControls`, `Text`, etc.), accelerate de comunitate, care au economisit timp de implementare.
- **Optimizări de performanță:** Datorită reconciler-ului React, R3F actualizează doar părțile din scenă care se schimbă.

Așadar, combinația React + React Router DOM + Tailwind CSS + React Three Fiber + Drei ne-a oferit un stack unificat, performant și ușor de întreținut, bine adaptat pentru dezvoltarea rapidă de simulări 3D interactive în browser și pentru extinderea ușoară a platformei VisioScience3D. Se poate observa în figura 4.7 cum arată arhitectura simplificată a frontend-ului platformei.

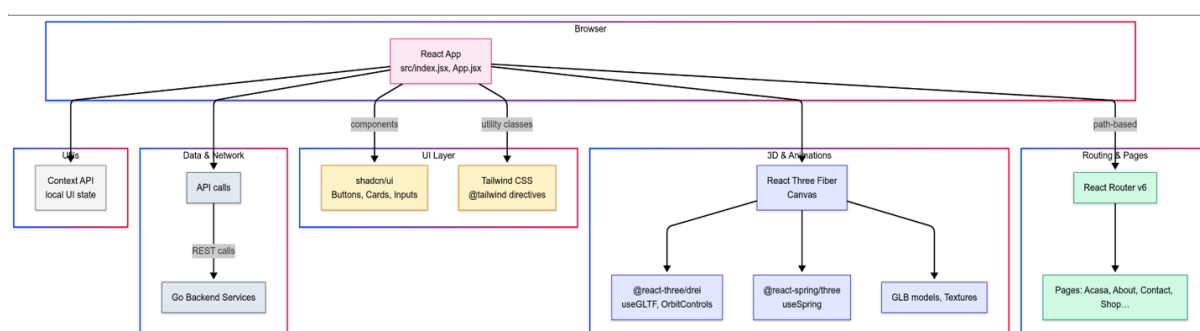


Figure 4.1: Diagrama de design principală a frontend-ului

### 4.1.2 Framework 3D

După cum am discutat anterior, frameworkul 3D ales este React Three Fiber, care este o bibliotecă care integrează Three.js cu React, permițându-ne să creăm scene 3D într-un mod declarativ și reactiv. Aceasta se bazează pe principii de WebGL combinate cu ce poate oferi React, cum ar fi componentizarea și gestionarea stării. R3F ne permite să creăm scene 3D complexe folosind JSX și să integrăm hook-uri ca:

- **useFrame:** Acesta permite să actualizăm scena la fiecare cadru, facilitând animațiile și interacțiunile în timp real.
- **useLoader:** Acesta ajută să încărcăm resurse externe (texturi, modele 3D) într-un mod eficient, folosind promisiuni pentru a gestiona încărcarea asincronă.

- **useThree:** Acesta oferă acces la obiectul Three.js curent, permițându-ne să interacționăm direct cu scena, camera și renderer-ul.
- **useRef:** Acesta permite să creăm referințe la obiectele din scenă, facilitând manipularea lor direct în timpul animațiilor sau interacțiunilor.
- **useState:** Acesta permite să gestionăm starea componentelor, facilitând actualizarea și redarea dinamică a obiectelor din scenă.
- **useEffect:** Acesta permite să gestionăm efectele secundare, cum ar fi adăugarea de evenimente sau actualizarea stării în funcție de modificările din scenă.

Pentru integrarea obiectelor cu extensia .glb am folosit un convertor gltf și anume [20] care ne permite să convertim modele .glb în componente React Three Fiber oferind grupuri de mesheuri și materiale. Acest lucru a permis să importăm modelele 3D direct în aplicația React și să creăm diverse manipulări și animații folosind API-ul R3F.



Figure 4.2: Exemplu de convertire a unui model 3D .glb în R3F

### 4.1.3 Modelare 3D

Am folosit Blender pentru a crea modelele 3D utilizând modele și primitive existente pe internet. De exemplu modelul paginii principale a fost făcut din primitive de insule și castele și adaptat la un număr de insule potrivit materiilor din meniu, cât și loc pentru suport viitor pentru alte materii.

Putem observa în figura următoare dezvoltarea modelului în Blender.

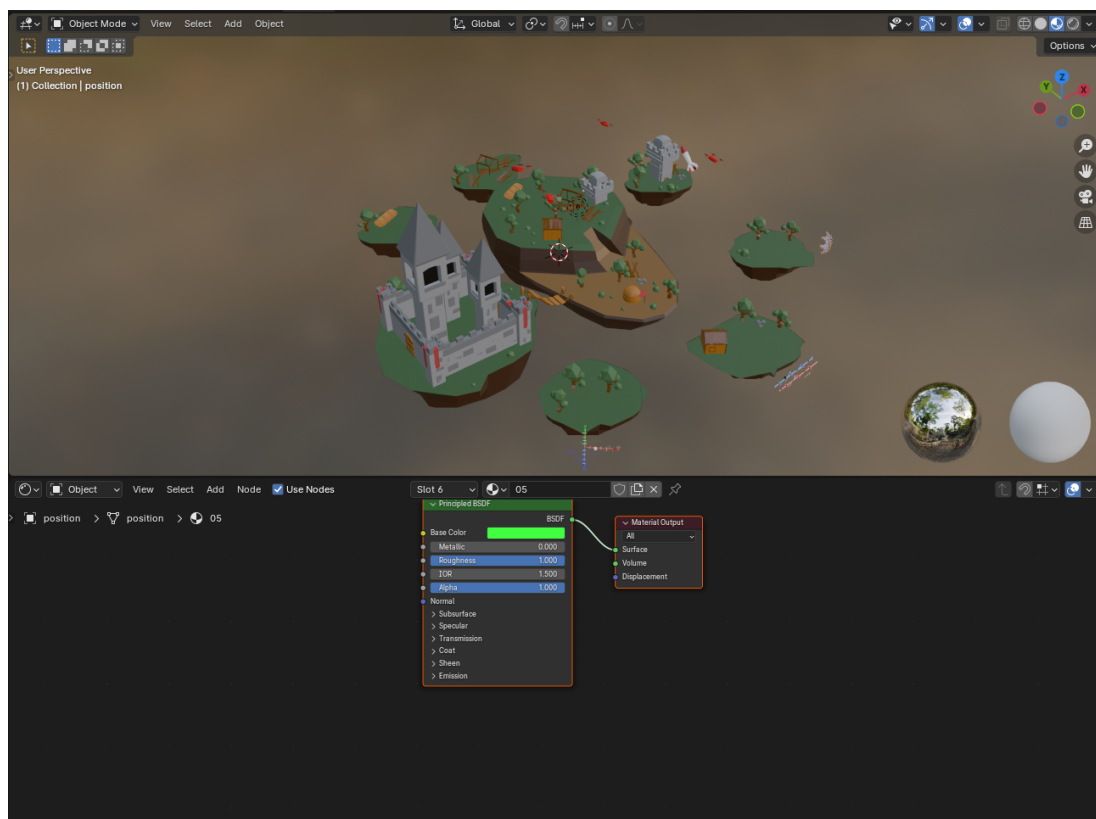


Figure 4.3: Modelul paginii principale în Blender

La acest model s-au adăugat animații de rotație și schimbare a state-ului la mișcarea mouse-ului, animație continuă de plutire și animații realiste de zbor pentru dronele din jurul insulelor.

## 4.2 Soluția UI/UX

### 4.2.1 Brandingul proiectului

#### Paleta de culori

Brandingul VisioScience3D se bazează pe o paletă caldă, prietenoasă și destul de pastelată, menită să creeze o atmosferă de joc și luminoasă pentru elevi. Fundalurile folosesc un gradient subtil de la #fdf4ff (alb-roz pal) prin #f3e8ff (lavandă delicată) spre #fff7ed (piersică pal), în timp ce culorile de accent dau personalitate interfeței: nuanța profundă de mulberry (#690375) marchează elementele active, bordurile și textele importante, iar tonul rosy-brown (#AE847E) individualizează secțiunile de chimie și astronomie. Pentru evidențierea vizualizărilor 3D am introdus un violet închis (#4f46e5). Culorile reflectă o atmosferă dinamică și energetică, încurajând explorarea și învățarea.

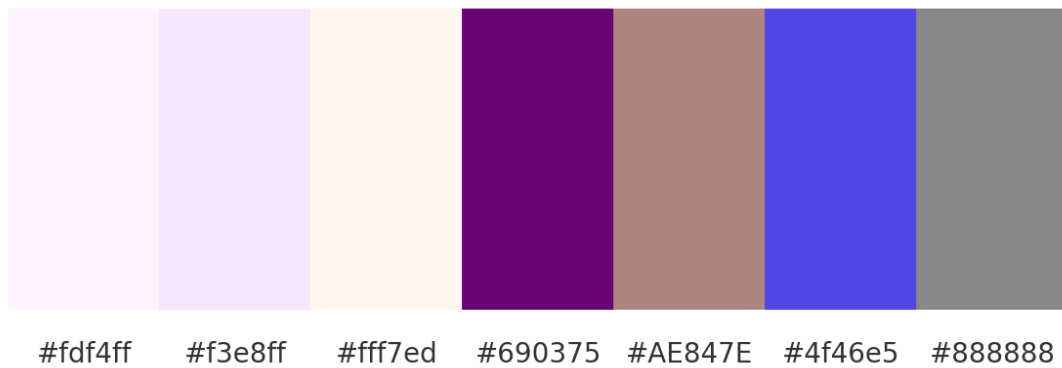


Figure 4.4: Paleta de culori a platformei

### Logo-ul

Logo-ul VisioScience3D este relativ simplu și în temă cu paleta de culori, fiind reprezentat de cuvântul "VisioScience3D" scris cu un font prezentativ și colorat cu un gradient de la mov la mulberry (#690375).

**VisioScience3D**

Figure 4.5: Logo-ul platformei



### Mascota platformei

Mascota proiectului este un balon care se apropie de paleta de culori a platformei și care simbolizează explorarea și învățarea. Balonul apare în toate paginile importante ale platformei și este de fiecare dată animat potrivit acțiunilor paginii respective. De exemplu, pe pagina de înregistrare mascota se mișcă în sus și în jos când utilizatorul scrie în câmpurile de înregistrare, iar pe la înregistrare reușită mascota sare în sus și în jos pentru câteva secunde. Balonul mascotă este și principalul caracter de explorare a meniului principal, învârtindu-se între insule pentru a ajunge la diferitele materii.



Figure 4.6: Mascota platformei

Se poate observa și o captură în timpul animației făcută la succesul la înregistrare.

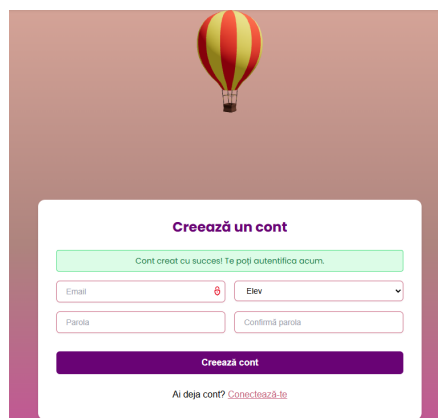


Figure 4.7: Captură de ecran a mascotei la înregistrare

### 4.2.2 Experiența utilizatorului

#### Navigarea în platformă

Navigarea în platformă este realizată printr-un meniu principal situat în partea de sus a paginii, iar în meniul 3D prin rotirea balonului mascotă în jurul insulelor pentru a ajunge la diferitele materii. În profil meniul funcționează pe bază de stări și se schimbă în funcție de pagina curentă.

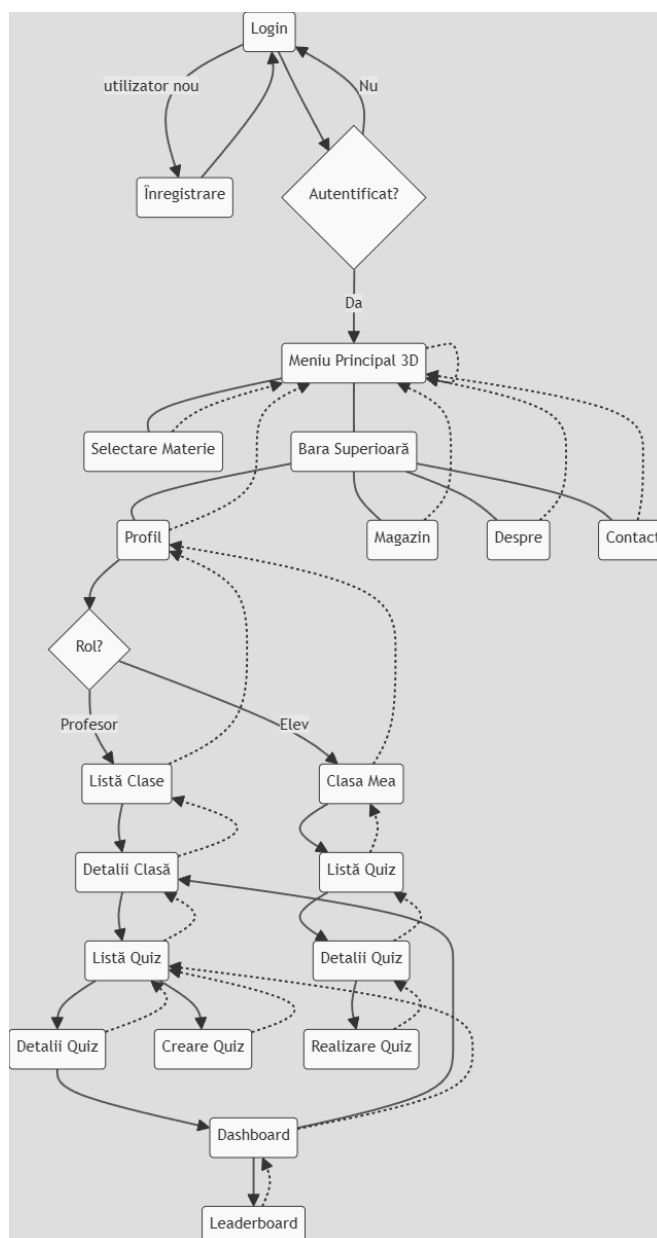


Figure 4.8: Diagrama de navigare a platformei

### Gestionarea stărilor

Gestionarea stărilor legate de autentificare și profilul utilizatorului se realizează prin stări locale (`useState`), efecte de ciclu de viață (`useEffect`) și persistență în `localStorage`:

- **Stări locale:** variabilele declarate cu `useState` păstrează răspunsul API-ului, semnul de încărcare și orice mesaj de eroare.
- **Efecte asincrone:** în `useEffect` se trimite cererea HTTP cu token-ul extras o singură dată din `localStorage`, iar la succes se apelează `setUser()`, iar la eșec `setError()`; indiferent de rezultat, în `finally` se setează `setLoading(false)`.
- **Persistență:** token-ul JWT și `userId` sunt citite și stocate o singură dată de la prima montare a componentei, ceea ce permite menținerea autentificării între reîncărcări.

Acest model este simplu dar se poate extinde cu *React Context* sau librării dedicate (`Redux`, `Zustand`) atunci când numărul de componente care împart aceleași stări crește semnificativ.

### Designul interfeței utilizatorului

Interfața păstrează un design simplu cu meniul plasat sus și în stânga pentru selecția materiilor după rotire.

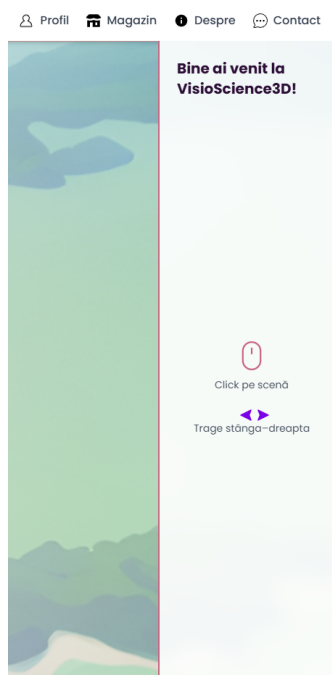


Figure 4.9: Interfața meniului principal

interfața profilului este simplă și ușor de folosit, având elemente care ies ușor în evidență și care sunt ușor de accesat cu acțiuni simple de tipul selecție, drag-and-drop sau butoane de toggle. De exemplu în imagine de mai jos se poate observa interfața profilului elevului.

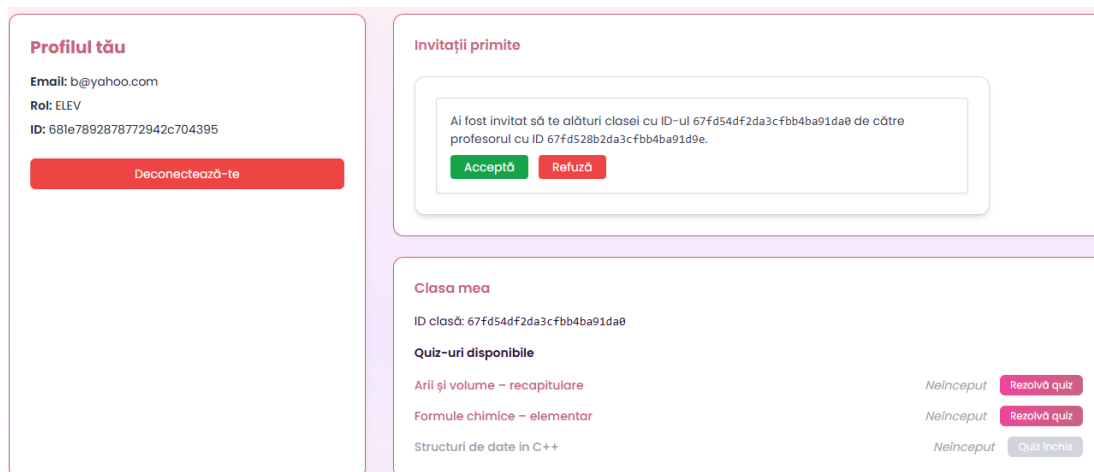


Figure 4.10: Interfața profilului elevului

## 4.3 Soluția de creare a scenelor 3D

### 4.3.1 Utilizarea tehnologiilor WebGL și Three.js

#### Crearea și gestionarea scenelor 3D

Scenele 3D se creează folosind tagul de tip Canvas din React Three Fiber, care ne permite să definim o zonă de desenare 3D în care putem adăuga obiecte, camere și lumini. Acestea sunt gestionate printr-o ierarhie de componente React, fiecare reprezentând un obiect 3D sau un grup de obiecte.

Se folosește tagul a din @react-spring/three care ne dă acces la grupuri de meshe-uri și materiale, permițându-ne să transformăm obiecte glb în jsx.

#### Interacțiunea cu obiectele 3D

Folosim useFrame pentru a actualiza scena la fiecare cadru, simulând callback-urile clasice din loop-urile din jocuri. UseEffect e utilizat să actualizeze starea componentelor în funcție de modificările din props, practic folosind addEventListener pentru a asculta evenimentele de schimbare a stării pe canvas sau document. UseRef, UseGLTF și UseThree au rolul de a crea referințe la obiectele din scenă, permițându-ne să le manipulăm direct în timpul animațiilor sau interacțiunilor. Folosim callbackuri de bază ca și stopPropagation pentru a preveni propagarea

evenimentelor mouse-ului de exemplu și `preventDefault` pentru a preveni comportamentele implicite.

### Optimizarea performanței graficii 3D

Pentru a optimiza performanța graficii 3D pe framework-ul bazat pe WebGL, se abordează câteva tehnici generale:

- **Reducerea numărului de draw calls:** Combinăm mesh-urile similare folosind `BufferGeometry` pentru a limita costul fiecărui cadru.
- **Frustum culling și occlusion culling:** WebGL elimină automat obiectele aflate în afara câmpului vizual, iar React Three Fiber expune aceste mecanisme nativ, astfel încât obiectele invizibile nu sunt trimise GPU-ului.

React Three Fiber (`react-three/fiber`) și Drei (`react-three/drei`) ne oferă setări și hook-uri dedicate pentru performanță:

- `<Canvas dpr=[1,1] />` limitează *device pixel ratio* la un interval controlat, prevenind supraîncărcarea GPU-ului pe ecrane cu densitate mare.
- Proprietatea `performance={min:0.5, max:1}` ajustează dinamic rata de reîmprospătare a canvas-ului în funcție de activitate, reducând numărul de cadre când scena este statică.
- Hook-ul `useTexture` din Drei preîncarcă și cache-uește texturile, evitând alocări repetate și blocări de I/O în bucla de randare.

În proiectul *VisioScience3D* am aplicat aceste principii astfel:

- Texturile sunt decupate și redimensionate la rezoluții de  $1k-2k$  și convertite în formate comprimate (JPEG/PNG cu bitrate optim), reducând traficul GPU.
- Geometria sferică a planetelor folosește un număr optim de segmente (32–64), echilibrând netezimea cu numărul de vârfuri procesate.
- Am folosit `React.memo` și `useMemo` pentru a nu recrea geometrii și materiale la fiecare rerender React, iar animațiile rulează exclusiv pe GPU prin `useFrame`.
- Setările WebGL `toneMapping` și `outputEncoding` sunt ajustate pentru a profita de hardware-ul mai slab și cel mobil cu `powerPreference: low-power`, asigurând stabilitate și consum redus de resurse.

Pentru a optimiza performanța graficii 3D, folosim texturi de dimensiuni mici și medii și

### 4.3.2 Integrarea cu frontendul

Pentru integrarea modelelor 3D în platformă, fiecare scenă sau obiect (.glb) este încărcat cu declanșatorul `useGLTF` din `react-three/drei` și convertit într-o componentă React care reacționează la stările de aplicație. De exemplu, componenta `Balloon` folosește un `useEffect` pentru a schimba periodic `animationState` într-o listă de stări (`idle`, `swayLeft`, `floatUp` etc.), iar declanșatorul `useSpring` din `react-spring/three` transformă această stare într-un set de proprietăți `position` și `rotation` animate, atribuite mai departe unui element `mesh`.

În plus, componenta de background atașează direct evenimente DOM (`mousedown`, `mousemove`, `keydown`) și gestionate în interiorul declanșatorului `useFrame` din `react-three/fiber`, permițând controlul direct al rotației scenei în funcție de interacțiunea utilizatorului. Starea și logica aplicației se propagă în lumea 3D, creând o leagătură reactivă între UI-ul convențional și grafica gestionată prin WebGL.

## Chapter 5

# Tehnologii utilizate pentru back-end

### 5.1 Descrierea arhitecturii

Backend-ul este construit folosind o arhitectură cu microservicii, care permite scalabilitate și întreținere ușoară a serviciilor care rulează. Avem trei servicii principale pentru logică de produs, un serviciu care funcționează ca router al arhitecturii, două instanțe de baze de date nerelaționale, un serviciu de gestionare a infrastructurii, un serviciu de monitorizare care funcționează împreună cu un serviciu de provizionare de metrice și un serviciu de vizualizare și utilitarele corespunzătoare pentru gestionarea bazelor de date integrate.

#### 5.1.1 Configurarea clusterului

Clusterul este configurat folosind Kubernetes, care permite orchestrarea containerelor și gestionarea resurselor într-un mod eficient. Fiecare serviciu este containerizat folosind Docker, permițându-ne să rulăm aplicațiile într-un mediu izolat și consistent. Kubernetes gestionează scalarea automată a serviciilor, distribuția sarcinilor și asigură disponibilitatea acestora.

Clusterul a fost creat utilizând kind (Kubernetes in Docker), care permite rularea Kubernetes într-un mediu de dezvoltare local. Acest lucru facilitează testarea și dezvoltarea aplicațiilor înainte de a le lansa în mediul de producție.

Dezvoltarea în acest mediu a permis integrarea rapidă a noilor funcționalități și testarea acestora într-un mediu controlat. De asemenea, a permis simularea unor scenarii de scalabilitate și performanță, asigurându-ne că aplicația poate gestiona un număr mare de utilizatori și cereri simultane.

Procesul de implementare și lansare presupune construirea imaginilor Docker pentru fiecare serviciu, încărcarea acestora în registrul de imagini gestionat prin Docker Desktop pentru testare și apoi reîncărcarea acestora în clusterul Kubernetes. Acest proces este automatizat prin intermediul unui pipeline CI/CD în mediul de dezvoltare Github. Pentru asta se folosesc GitHub Actions, care permit rularea automată a diferitelor procese de construire, formatare și apoi trecerea prin procesul de testare și implementare/lansare a aplicației.

Fiecare serviciu are propriile fișiere *.yaml* de configurare, care definesc care definesc după caz modul de rulare, variabilele de mediu, resursele necesare, rutarea, porturile expuse și alte setări specifice. Aceste fișiere sunt esențiale pentru a asigura funcționarea corectă a serviciilor în cadrul clusterului Kubernetes și pentru a permite gestionarea eficientă a acestora.

### 5.1.2 Structurarea și crearea fișierelor YAML

Pentru fiecare componentă a backend-ului—Deployment, Service, Ingress, ConfigMap, PersistentVolumeClaim — s-au creat fișiere YAML separate în directoarele *./k8s/* ale fiecărui serviciu. Denumirea fișierelor corespunde resursei Kubernetes:

- **\*-deployment.yaml** pentru kind: Deployment
- **\*-service.yaml** pentru kind: Service
- **\*-ingress.yaml** pentru kind: Ingress
- **\*-configmap.yaml** pentru kind: ConfigMap
- **\*-pvc.yaml** pentru kind: PersistentVolumeClaim

Au fost aplicate toate manifesturile cu:

```
1 kubectl apply -f ./k8s/
```



Un exemplu de *Deployment* pentru serviciul de evaluare:

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: evaluation-service
5   labels:
6     app: evaluation-service
7 spec:
8   replicas: 1
9   selector:
10    matchLabels:
11      app: evaluation-service
12   template:
13     metadata:
14       labels:
15         app: evaluation-service
16     spec:
17       containers:
18         - name: evaluation-service
19           image: dragomir1401/evaluation-service:latest
20           ports:
21             - containerPort: 8080
22           env:
23             - name: MONGO_URI
24               value: "mongodb://mongo-evaluation-service:27017"
```

Acesta definește un *Deployment* pentru serviciul specificat, cu un singur replica pentru rulare și specifică imaginea Docker care va fi utilizată. De asemenea, definește variabilele de mediu necesare pentru conectarea la baza de date. Este unul dintre exemplele simple de manifest Kubernetes folosit.

**Tipuri de resurse și rolurile lor**

- **Service** (kind: Service): expune pod-urile pe un IP intern al clusterului:

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: evaluation-service
5  spec:
6    type: ClusterIP
7    selector:
8      app: evaluation-service
9    ports:
10     - port: 8080
11       targetPort: 8080
```

- **Ingress** (kind: Ingress): definește rutare HTTP/S prin controller folosit (Kong):

```
1  apiVersion: networking.k8s.io/v1
2  kind: Ingress
3  metadata:
4    name: evaluation-service-ingress
5    annotations:
6      konghq.com/strip-path: "false"
7  spec:
8    ingressClassName: kong
9    rules:
10     - http:
11       paths:
12         - path: /evaluation
13           backend:
14             service:
15               name: evaluation-service
16               port:
17                 number: 8080
```

- **ConfigMap & PVC:** ConfigMap pentru configurări (de ex. pentru serviciul de provizionare `prometheus.yml`) și PVC pentru volumul serviciului de vizualizare a metricilor Grafana:

```
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: prometheus-config
5  data:
6    prometheus.yml: |
7      global:
8        scrape_interval: 15s
9        evaluation_interval: 15s
10 ---
11 apiVersion: v1
12 kind: PersistentVolumeClaim
13 metadata:
14   name: grafana-pvc
15 spec:
16   accessModes: [ReadWriteOnce]
17   resources:
18     requests:
19       storage: 1Gi
```

Fiecărei resurse îi corespunde un singur fișier YAML – conținând `apiVersion`, `kind`, meta-datele și `spec` – care poate fi aplicat cu **kubectl apply -f <fișier>**, sau automatizat printr-un pipeline de integrare continuă / livrare continuă (CI/CD).

### 5.1.3 Gestionarea clusterului

Clusterul este creat și gestionat în principal cu Kubernetes, folosind manifesturi YAML. Pe lângă linia de comandă, am integrat *Portainer CE* pentru o interfață web unificată de administrare. Portainer rulează ca un Deployment sub contul de serviciu `portainer-sa`, căruia i s-a atribuit un ClusterRole cu toate privilegiile și un ClusterRoleBinding aferent. Manifestele corespunzătoare (`portainer-sa`, `portainer-role`, `portainer-deployment`, `portainer-rolebinding`, `portainer-service` și `portainer-ingress`) sunt folosite pentru configurare științelor gestionare accesului Portainer în cluster. Interfața Portainer, expusă printr-un Service și un Ingress pe ruta `/portainer`, permite vizualizarea nodurilor,

pod-urilor, volumelor și gestionarea configurațiilor (scaling, redeploy, logs) fără comenzi complexe, oferind în același timp controale RBAC și monitorizare centralizată.

Se poate vedea în imaginea de mai jos cum arată interfața Portainer pe clusterul nostru:

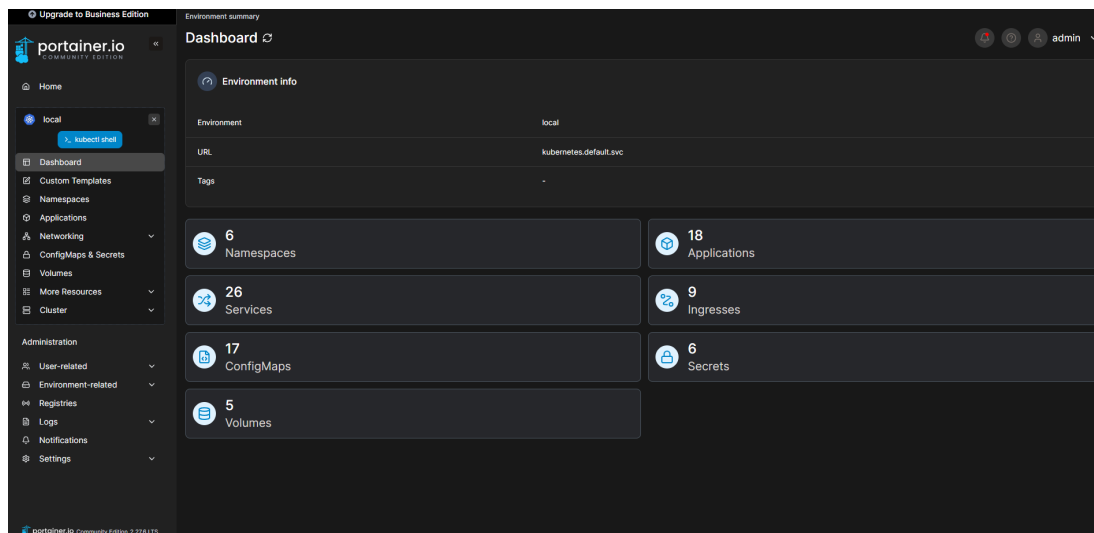


Figure 5.1: Interfața Portainer pe clusterul nostru

Și mai jos se poate observa listarea serviciilor în linie de comandă versus în Portainer:

| Service                 | Application        | Namespace | Type         | Ports          | Target Ports | Cluster IP    | External IP | Created             |
|-------------------------|--------------------|-----------|--------------|----------------|--------------|---------------|-------------|---------------------|
| evaluation-service      | evaluation-service | default   | ClusterIP    | 8080/TCP       | 8080         | 10.96.174.155 | -           | 2020-04-21 14:08:00 |
| feed-data               | feed-data          | default   | ClusterIP    | 8080/TCP       | 8080         | 10.96.158.202 | -           | 2020-03-30 11:53:02 |
| grafana                 | grafana            | default   | ClusterIP    | 8080/TCP       | 8080         | 10.96.201.226 | -           | 2020-03-28 17:53:39 |
| kong                    | kong               | default   | ClusterIP    | 3000/TCP       | 3000         | 10.96.202.250 | -           | 2020-03-13 23:24:14 |
| kong-controller         | kong-controller    | kong      | ClusterIP    | 10259/TCP      | 10259        | 10.96.158.208 | -           | 2020-03-30 14:59:31 |
| kong-controller-webhook | kong-controller    | kong      | ClusterIP    | 443/TCP        | webhook      | 10.96.202.04  | -           | 2020-03-30 14:59:31 |
| kong-gateway            | kong-gateway       | kong      | ClusterIP    | 8444/TCP       | 8444         | None          | -           | 2020-03-30 14:59:31 |
| kong-gateway-admin      | kong-gateway       | kong      | NodePort     | 8000-34465/TCP | 8000         | 10.96.202.37  | -           | 2020-03-30 14:59:31 |
| kong-gateway-manager    | kong-gateway       | kong      | NodePort     | 8443-8257/TCP  | 8443         | 10.96.202.37  | -           | 2020-03-30 14:59:31 |
| kong-gateway-proxy      | kong-gateway       | kong      | LoadBalancer | 80-8080/TCP    | 8080         | 10.96.21.81   | -           | 2020-03-30 14:59:31 |
| kong-kong               | kong-kong          | default   | NodePort     | 8000-34465/TCP | 8000         | 10.96.202.37  | -           | 2020-03-30 14:59:31 |
| kong-kong-manager       | kong-kong          | default   | NodePort     | 8443-8257/TCP  | 8443         | 10.96.202.37  | -           | 2020-03-30 14:59:31 |

| NAME                                     | READY | STATUS  | RESTARTS      | AGE  |
|------------------------------------------|-------|---------|---------------|------|
| evaluation-service-7f9fbf4774-t8mqc      | 1/1   | Running | 0             | 7d6h |
| feed-data-85dc88d4c4-fm9xc               | 1/1   | Running | 1 (7d6h ago)  | 18d  |
| grafana-d6c6c7876-7lmv9                  | 1/1   | Running | 0             | 7d6h |
| kong-kong-847bf5db6-vqzjn                | 2/2   | Running | 40 (7d6h ago) | 31d  |
| mongo-express-feed-data-56959db6f8-wj5nn | 1/1   | Running | 11 (7d6h ago) | 31d  |
| mongo-express-user-data-6bb8d98bf4-h8fhh | 1/1   | Running | 15 (7d6h ago) | 56d  |
| mongo-feed-data-service-655758fd46-l77bx | 1/1   | Running | 16 (7d6h ago) | 56d  |
| mongo-user-data-service-59bb79895-edvpt  | 1/1   | Running | 16 (7d6h ago) | 56d  |
| monitoring-service-7597c657b5-phmd       | 1/1   | Running | 1 (7d6h ago)  | 19d  |
| portainer-b59849bbf-8wv6n                | 1/1   | Running | 0             | 7d5h |
| prometheus-6f457d9877-rv94z              | 1/1   | Running | 4 (7d6h ago)  | 7d6h |
| user-data-694498d699-w8tgc               | 1/1   | Running | 0             | 7d6h |

(a) Serviciile în Portainer

(b) Serviciile în linie de comandă

Figure 5.2: Compararea vizualizării serviciilor în Portainer și în linie de comandă

### 5.1.4 Rutarea și gestionarea traficului

Pentru a centraliza și controla traficul HTTP/S, am ales Kong ca API Gateway. În cluster am definit o clasă de rutare dedicată, astfel:

```
1 kind: IngressClass
2 metadata:
3   name: kong
4 spec:
5   controller: konghq.com/ingress-controller
```

Controller-ul Kong va intercepta toate resursele de tip Ingress care specifică `ingressClassName: kong`. În mediul de dezvoltare, pentru a nu expune un LoadBalancer extern, am folosit port-forwarding local:

```
kubect1 port-forward svc/kong-kong-proxy 8000:80
```

Astfel, apelurile HTTP către <http://localhost:8000> sunt tunelate prin Kong și pot fi testate rapid împotriva serviciilor din cluster fără configurări suplimentare de rețea.

## 5.2 Descrierea serviciilor

În continuare sunt listate serviciile care alcătuiesc backend-ul aplicației, fiecare având roluri și responsabilități specifice în cadrul arhitecturii microservicii.

- `evaluation-service` – Serviciul pentru gestionarea logicii de creare, monitorizare, deschide/închidere, ștergere, gestionare de teste și evaluări.
- `feed-data` – Serviciul care se ocupă de gestionarea datelor de feed-urile educaționale și accesarea motorului de generare a moleculelor de chimie.
- `grafana` – Grafana pentru vizualizarea metricilor și a datelor de monitorizare.
- `kong-kong` – Controller Kong pentru gestionarea rutării și a API-urilor.
- `mongo-express-feed-data` – Interfață web pentru gestionarea bazei de date a feed-urilor educaționale.
- `mongo-express-user-data` – Interfață web pentru gestionarea bazei de date a utilizatorilor.
- `mongo-feed-data-service` – Baza de date MongoDB pentru serviciul de feed-uri educaționale.
- `mongo-user-data-service` – Baza de date MongoDB pentru serviciul de utilizatori.
- `monitoring-service` – Serviciul de monitorizare care colectează și expune metrici.
- `portainer` – Interfață de administrare a clusterului Kubernetes.
- `prometheus` – Serviciul de colectare a metricilor și monitorizare prin Prometheus.
- `user-data` – Serviciul pentru gestionarea datelor utilizatorilor, inclusiv autentificare și autorizare.

### 5.2.1 Descrierea API-urilor

Mai jos sunt principalele grupuri de endpoint-uri fără a le include pe cele de ștergere din rațiuni de dimensiuni.

Table 5.1: Endpoint-uri REST ale microserviciilor

| Endpoint                                          | Descriere                   |
|---------------------------------------------------|-----------------------------|
| <b>Feed Data Service</b>                          |                             |
| POST /feed                                        | creare feed de formule      |
| GET /feed/{id}                                    | folosire feed de formulă    |
| PUT /feed/{id}                                    | actualizare feed de formulă |
| GET /feed/shape/{shape}                           | listare după scenă          |
| GET /feed/chem/molecules                          | listare molecule            |
| POST /feed/chem/molecules                         | creare moleculă             |
| GET /feed/chem/molecules/{id}                     | citire moleculă             |
| PUT /feed/chem/molecules/{id}                     | actualizare moleculă        |
| GET /feed/chem/molecules/{id}/3d                  | vizualizare 3D moleculă     |
| <b>User Data Service</b>                          |                             |
| POST /user/auth/register                          | înregistrare utilizator     |
| POST /user/auth/login                             | autentificare utilizator    |
| GET /user/me                                      | detalii profil curent       |
| POST /user/classes                                | creare clasă                |
| GET /user/classes                                 | listare clase proprii       |
| POST /user/classes/{code}/join                    | aderare la clasă            |
| POST /user/class/add-student                      | adaugă elev                 |
| GET /user/classes/{id}/students                   | listează elevi              |
| POST /user/classes/{id}/invite                    | trimite invitație           |
| GET /user/invites                                 | listare invitații           |
| POST /user/invites/{id}/respond                   | răspunde invitației         |
| POST /user/quiz/result                            | trimite rezultat quiz       |
| GET /user/quiz/results/{quizId}                   | rezultate individuale       |
| GET /user/classes/{classId}/quiz/{quizId}/results | rezultate pe clasă          |
| <b>Evaluation Service</b>                         |                             |
| POST /evaluation/quiz                             | creare quiz                 |
| GET /evaluation/quiz                              | listare quiz-uri            |
| PUT /evaluation/quiz/{id}                         | actualizare quiz            |
| GET /evaluation/quiz/class/{class_id}             | listare pe clasă            |
| GET /evaluation/quiz/attempt/{quizId}             | pregătire quiz              |
| POST /evaluation/quiz/attempt/{quizId}            | trimitere răspunsuri        |
| GET /evaluation/quiz/{quizId}/result/{userId}     | ultim rezultat              |
| GET /evaluation/quiz/{quizId}/results             | rezultate multiple          |
| PUT /evaluation/quiz/{id}/status                  | schimbă status quiz         |

Fiecare serviciu are un API RESTful care permite interacțiunea cu resursele sale. Aceste API-uri sunt definite în routerul fiecărui serviciu care este creat prin biblioteca Gorilla Mux în Go. Fiecare serviciu are un set de endpoint-uri care permit operații CRUD (Create, Read, Update, Delete) asupra resurselor sale și interacționează cu baza de date corespunzătoare.

## 5.3 Securitate

Toate API-urile serviciilor sunt protejate prin JWT, validat de middleware-ul JWTAuth, care respinge cererile fără token sau cu token invalid. Traficul extern este dirijat prin Kong, configurat cu HTTPS/TLS și politici de rate-limiting, CORS și strip-path pentru a preveni atacurile de tip „path traversal” și abuzul de resurse. În interiorul clusterului, rolurile și permisiunile sunt gestionate prin RBAC Kubernetes (ClusterRole/ClusterRoleBinding), iar Portainer rulează sub un ServiceAccount cu drepturi strict necesare. Conexiunile la bazele MongoDB sunt realizate folosind URI-urile securizate (MONGO\_URI) și, în producție, se va face activarea criptării TLS și a autentificării la nivel de server de baze de date.

## 5.4 CI/CD

### 5.4.1 Port forwarding

În mediul de dezvoltare, pentru a testa rapid Ingress-ul Kong fără LoadBalancer extern, se folosește port-forwarding:

```
1 kubectl port-forward svc/kong-kong-proxy 8000:80
```

Apelurile la <http://localhost:8000> trec prin Kong și pot fi rutate către microservicii.

### 5.4.2 Deployment

Am automatizat build-ul, testarea și deploy-ul în Kubernetes cu GitHub Actions. Pe scurt:

- **Checkout & Context:** selectăm codul pentru fiecare serviciu și setăm contextul kubectl către kind-visioscience, clusterul unde rulează local mediul de dezvoltare.
- **Build & Push Docker:** compilăm binarele Go, construim imaginea cu build-push-action și o încărcăm pe DockerHub sub dragomir1401/\$SERVICIU:latest.
- **Deploy:** rulăm:

```
1 kubectl apply -f k8s/  
2 kubectl rollout status deployment/${{ steps.extract.outputs.name }}
```

- Runner-ul GitHub este un self-hosted pe aceeași mașină ca și clusterul Kind, astfel că are acces direct la kubeconfig.

### 5.4.3 Metrice / Monitorizare

#### 5.4.4 Avantajele folosirii Prometheus

Prometheus este un sistem de monitorizare și alertare foarte popular în mediile moderne de dezvoltare și producție datorită scalabilității și flexibilității sale. Oferă o arhitectură simplă bazată pe colectarea activă a metricilor prin intermediul unor endpoint-uri HTTP expuse de aplicații (format `exposition format`), facilitând integrarea rapidă cu o varietate largă de servicii și microservicii. În plus, arhitectura sa detașată de stare (stateless) și suportul pentru stocarea pe termen scurt cu export către sisteme externe îl fac ideal pentru arhitecturi dinamice, precum Kubernetes. De asemenea, se integrează nativ cu instrumente vizuale precum Grafana, oferind dashboard-uri ușor de configurat și interpretat de către echipele tehnice. În cazul platformei dezvoltate Grafana a recunoscut sursa de date din parte Prometheus direct fără configurări suplimentare și s-au putut crea panourile de vizualizare direct.

#### 5.4.5 Colectare Prometheus

În fiecare serviciu Go am folosit clientul oficial Prometheus:

- [github.com/prometheus/client\\_golang/prometheus](https://github.com/prometheus/client_golang/prometheus) pentru definirea contorilor, histogramei și gauge-urilor.
- [github.com/prometheus/client\\_golang/prometheus/promhttp](https://github.com/prometheus/client_golang/prometheus/promhttp) pentru expunerea endpoint-ului `/${SERVICIU}/metrics`.

De exemplu, în `evaluation-service` în `metrics/metrics.go`:

```
1 registry := prometheus.NewRegistry()
2 HTTPRequestsTotal := prometheus.NewCounterVec(...)
3 registry.MustRegister(HTTPRequestsTotal, HTTPRequestDuration,
   ↪ ActiveEvaluations)
4 http.Handle("/evaluation/metrics", promhttp.HandlerFor(registry, promhttp.
   ↪ HandlerOpts{}))
```

#### 5.4.6 Grafana Dashboard

De exemplu după ce Prometheus scrapează `evaluation_service_http_requests_total`, se crează un dashboard persistent prin volumele Kubernetes. Dashboard-ul Grafana are în general două tipuri de panouri pentru metrice similare. De exemplu pentru metrica de cereri și



evaluări active din serviciul de evaluare:

- *Timeseries* pentru `evaluation_service_http_requests_total`
- *Gauge* pentru `evaluation_service_active_evaluations`

Un fragment din JSON-ul dashboard-ului Grafana arată astfel:

```

1  "panels": [
2    {
3      "type": "timeseries",
4      "targets": [{"expr": "evaluation_service_http_requests_total"}],
5      "title": "HTTP_Requests"
6    }, {
7      "type": "gauge",
8      "targets": [{"expr": "evaluation_service_active_evaluations"}],
9      "title": "Active_Evaluations"
10   }
11 ]

```

Aceste panouri sunt importate în Grafana sub `Evaluation Service Dashboard` și actualizate la fiecare 5s (configurabil).

Așadar există trei panouri principale în Grafana pentru trei servicii cu logică de produs existente:

- `Evaluation Service Dashboard` pentru `evaluation-service`

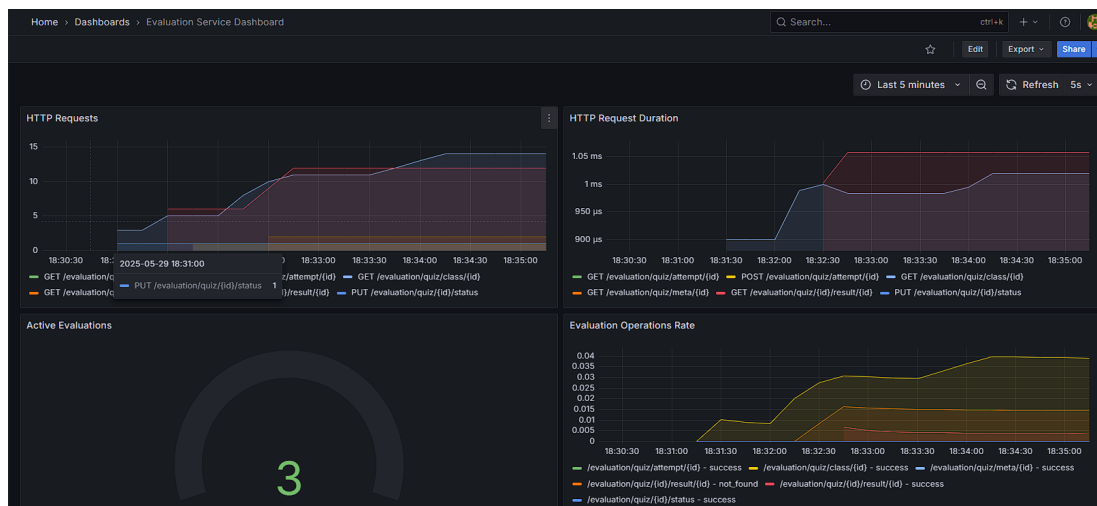


Figure 5.3: Dashboard-ul pentru evaluări în Grafana

Se poate observa logarea numărului de cereri HTTP din serviciul, latența și durată medie

a lor, numărul de evaluări active și rata operațiilor pe evaluări.

- Feed Data Service Dashboard pentru feed-data

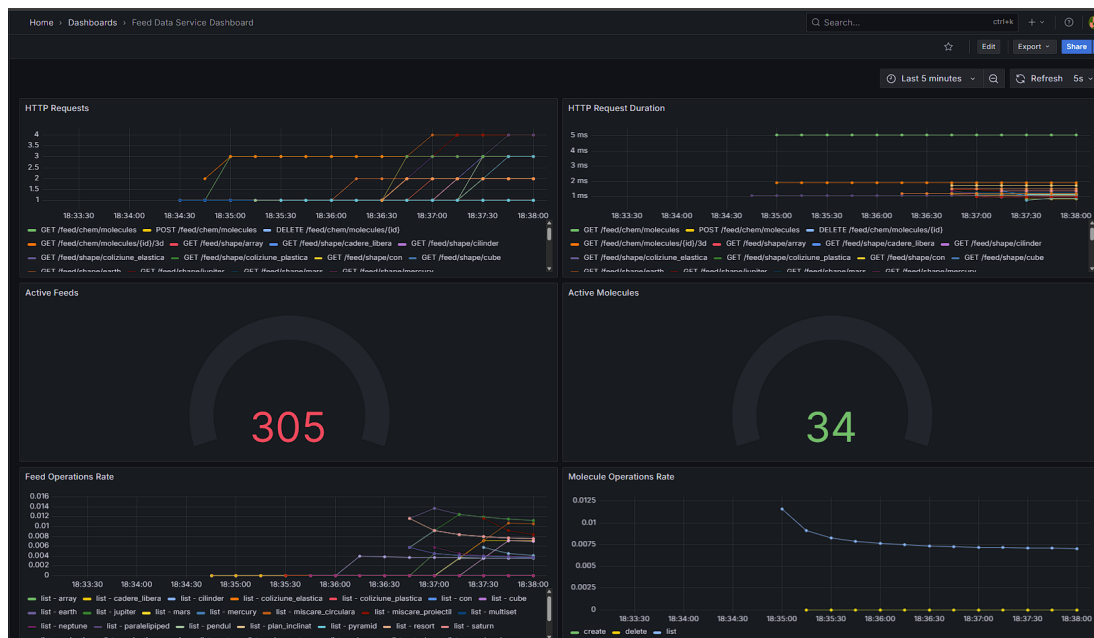


Figure 5.4: Dashboard-ul pentru metrice de feed-uri în Grafana

În panou se pot vedea numărul de cereri HTTP, latența și durata medie a lor, numărul de feed-uri active, numărul de molecule active, rata operațiilor asupra feed-urilor și moleculelor, precum și numărul de molecule generate.

- User Data Service Dashboard pentru user-data



Figure 5.5: Dashboard-ul pentru metrice de utilizator în Grafana

Apar numărul, latența și durata cererilor HTTP, numărul utilizatorilor și claselor active.

## 5.5 Soluția de baze de date

### 5.5.1 Structura bazei de date

#### Tipuri de baze de date utilizate / Motivația alegerii

Baza de date folosită este MongoDB, o bază de date nerelațională care oferă flexibilitate și scalabilitate și e ușor de folosit și mai ales de integrat în microserviciile gestionate în Kubernetes. Am ales o soluție nerelațională deoarece datele noastre sunt semi-structurate și nu necesită relații complexe între tabele, permițându-ne să stocăm documente JSON care pot fi ușor extinse și modificate fără a necesita migrări de baze de date. Structura ar fi putut fi integrată și într-un mediu relațional, dar am considerat că MongoDB oferă o flexibilitate mai mare pentru tipurile de date pe care le gestionăm, cum ar fi feed-urile educaționale, datele utilizatorilor, cât și pentru structura testelor și evaluărilor.

Un alt avantaj al folosirii MongoDB este că permite stocarea datelor într-o formă orientată pe documente similare cu JSON. Alegerea s-a bazat și pe flexibilitatea schemelor, scalabilitatea orizontală și compatibilitatea nativă cu structurile de date folosite, precum și pe posibilitatea de a face agregări complexe și de a gestiona relații parțiale între documente.

MongoDB permite modificarea ușoară a modelelor de date în timp, fără a impune o schemă fixă, aspect important pentru aplicații aflate în dezvoltare continuă și extindere.

#### Gestionarea datelor

Datele sunt organizate în colecții care corespund entităților sistemului: quiz-uri, formule, molecule, utilizatori, clase și invitații. Fiecare colecție conține documente ce respectă structurile definite în codul backend-ului, folosind driverul oficial MongoDB pentru Go ([go.mongodb.org/mongo-driver](https://go.mongodb.org/mongo-driver)).

De exemplu, un document din colecția `quizzes` conține câmpuri precum `Title`, `ClassID` și `OwnerID` (reprezentate prin `ObjectID` MongoDB), un array de `Questions` cu întrebări, răspunsuri și punctaje, și un array `QuizResults` pentru stocarea scorurilor utilizatorilor.

Se folosesc structuri Go care reflectă aceste modele, permițând deserializarea și serializarea automată între BSON și JSON. Astfel se poate realiza interacțiunea cu baza se face prin operații CRUD folosind metodele MongoDB standard, iar aplicația transformă și face validarea datelor ușor.

### 5.5.2 Securitatea datelor

Datele stocate sunt protejate prin controlul accesului la nivel de cluster MongoDB, autentificare și autorizare prin roluri definite (de exemplu, utilizator, profesor, administrator). Parolele sunt criptate și gestionate prin variabile de mediu la nivelul serviciilor. Pe viitor se poate face integrarea cu un serviciu de autentificare centralizat (ex: OAuth2, OpenID Connect) sau folosirea unui serviciu de identitate și secrete (precum HashiCorp Vault sau AWS Secrets Manager) pentru a gestiona securitatea și accesul și mai granular la datele sensibile.

### Performanța și scalabilitatea bazei de date

MongoDB permite scalarea orizontală prin sharding, facilitând distribuția datelor pe mai multe noduri când cantitatea datelor ar ajunge la dimensiuni mari. Indecșii sunt definiți pe câmpuri critice (ex: `_id`, `class_id`) pentru a optimiza căutările și interogările.

De asemenea, conexiunile sunt gestionate eficient prin pool-uri în clientul Go, iar operațiile lungi sunt efectuate cu timeout-uri pentru a evita blocarea serviciilor.

### 5.5.3 Integrarea cu back-endul

Back-endul este implementat în Go, folosind pachetul oficial MongoDB driver, care oferă suport complet pentru operațiile pe colecții și documente, tranzacții și agregări.

La pornire, fiecare serviciu inițializează o conexiune persistentă la baza de date. Modelele Go definite permit maparea clară între structurile din cod și documentele MongoDB. De exemplu, în `evaluation-service`, obiectele `Quiz` și `QuizResult` sunt manipulate direct în cod, iar metodele HTTP expun API REST care gestionează aceste resurse prin CRUD.

Așadar arhitectura descrisă permite un flux clar și mai ales scalabil între date și servicii, ușurând mentenanța serviciilor și extinderea aplicației.

## Chapter 6

# Detalii de implementare

În această secțiune se va discuta despre implementarea platformei, acoperind back-endul, front-endul, configurarea, dezvoltarea și conectivitatea acestora.

### 6.1 Back-end

Pentru back-end s-au analizat diferite tehnologii bazat pe comparația:

Table 6.1: Compararea limbajelor de programare care ar fi fost potrivite pentru backend-ul proiectului

| Caracteristică                  | Go                                                                                    | Node.js (JavaScript)                                            | Python                                                                                 |
|---------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>Performanță</b>              | Compilat, foarte rapid, cu suport nativ pentru concurență eficientă (goroutine)       | Interpretat, bun pentru I/O non-blocant, dar mai lent decât Go  | Interpretat, mai lent, CPU-intensive tasks pot fi mai lente                            |
| <b>Concurență</b>               | Goroutines ușor de folosit, eficiente din punct de vedere al resurselor               | Model event-loop, asincron cu callback-uri/promises/async-await | Suport limitat (threading, asyncio), mai complex pentru concurență                     |
| <b>Simplitate și claritate</b>  | Sintaxă simplă, tipare clare, cod ușor de întreținut                                  | Flexibil, dar poate duce la cod greu de urmărit (callback hell) | Sintaxă clară, dar structuri mari pot deveni greu de gestionat                         |
| <b>Ecosistem</b>                | Ecosistem în creștere, multe librării pentru backend, suport bun pentru REST, MongoDB | Ecosistem foarte matur și extins, multe librării, framework-uri | Ecosistem matur, multe librării pentru ML, backend, dar nu neapărat pentru performanță |
| <b>Scalabilitate</b>            | Foarte scalabil, folosit în microservicii și sisteme distribuite                      | Scalabil pe I/O, dar single-threaded cu limite pe CPU           | Scalabilitate limitată, se bazează pe external scaling și caching                      |
| <b>Deploy și containerizare</b> | Binar unic compilat, ușor de deployat în containere                                   | Necesită runtime Node.js, imagini mai mari                      | Necesită runtime Python și librării, imagini relativ mari                              |

Alegerea a fost influențată de mai mulți factori, printre care se numără performanța, ușurința de utilizare și suportul extins pentru dezvoltarea Web . Go oferă un model de concurență eficient și o sintaxă simplă, ceea ce îl face potrivit pentru construirea rapidă de aplicații scalabile. Alt motiv principal a fost compatibilitatea cu infrastructura aleasă pentru proiect, bazată pe Kubernetes și Docker, care se integrează foarte ușor cu Go.

### 6.1.1 Configurare back-end

Fiecare microserviciu are propria configurație Kubernetes definită prin fișiere YAML pentru Deployment, Service, ConfigMap și alte resurse necesare. Configurarea constă în containerizarea și automatizarea livrării folosind Docker pentru containerizarea serviciilor și GitHub Actions pentru lanțul CI/CD, care automatizează build-ul, testarea, și deploy-ul pe clusterul Kubernetes local.

Configurarea conexiunilor la baza de date MongoDB este centralizată în helperi, folosind driver-ul oficial MongoDB pentru Go. Monitorizarea serviciilor este asigurată prin integrarea Prometheus, cu metrice personalizate definite în cod și expuse la endpoint-uri REST.

---

```
1 func prometheusMiddleware(next http.Handler) http.Handler {
2     return http.HandlerFunc(func(w http.ResponseWriter, r *http.
        Request) {
3         if r.URL.Path == "/feed/metrics" {
4             next.ServeHTTP(w, r)
5             return
6         }
7         start := prometheus.NewTimer(metrics.HTTPRequestDuration.
            WithLabelValues(r.Method, utils.NormalizePath(r.URL.Path)
            ))
8         next.ServeHTTP(w, r)
9         start.ObserveDuration()
10        metrics.HTTPRequestsTotal.WithLabelValues(r.Method, utils.
            NormalizePath(r.URL.Path), "200").Inc()
11    })
12 }
```

---

Listing 6.1: Middleware Prometheus pentru monitorizarea duratei cererilor HTTP

### 6.1.2 Dezvoltare back-end

Codul este organizat în pachete separate pentru rute, modele de date, pachet cu fișiere ajutoare, metrice, utils, panou grafana, pachet cu fișiere Kubernetes de configurare și middleware (ex: autentificare JWT).

Se utilizează biblioteca Gorilla Mux pentru rutare HTTP, iar fiecare endpoint implementează operații CRUD pentru resurse precum quizuri, clase, molecule, formulare.

Validarea datelor și gestionarea erorilor sunt tratate consistent, cu înregistrare de metrice pentru succes și eșecuri, ceea ce permite monitorizarea calității serviciilor.

Pentru interacțiunea cu MongoDB, structurile Go sunt mapate la documente BSON, cu chei explicite și suport pentru ID-uri generate automat. Structurile BSON sunt convertite nativ în JSON pentru serializare/deserializarea datelor și trimiterea lor între servicii și front-end.

---

```
1 func GenerateToken(userID, role, email string) (string, error) {
2     claims := CustomClaims{
3         UserID: userID,
4         Role:    role,
5         Email:   email,
6         RegisteredClaims: jwt.RegisteredClaims{
7             ExpiresAt: jwt.NewNumericDate(time.Now().Add(24 * time.
                        Hour)),
8             IssuedAt:  jwt.NewNumericDate(time.Now()),
9         },
10    }
11    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
12    return token.SignedString(getJwtSecret())
13 }
14
15 func ParseToken(tokenString string) (*CustomClaims, error) {
16     token, err := jwt.ParseWithClaims(tokenString, &CustomClaims{},
17         func(token *jwt.Token) (interface{}, error) {
18             return getJwtSecret(), nil
19         })
20     if err != nil || !token.Valid {
```

```
20         return nil, errors.New("invalid_token")
21     }
22     return token.Claims.(*CustomClaims), nil
23 }
```

---

Listing 6.2: Generare și validare token JWT

### 6.1.3 Conectivitate

Microserviciile comunică prin API REST, iar traficul este expus și rulat în interiorul clusterului Kubernetes. Pentru dezvoltare locală, se utilizează port-forwarding Kubernetes pentru a expune temporar serviciile externe către mașina locală. Gestionarea rutelor externe și balanșării numărului de cereri se realizează cu ajutorul Kong Ingress Controller, care asigură rutarea traficului HTTP către serviciile potrivite.

---

```
1 func CreateQuiz(w http.ResponseWriter, r *http.Request) {
2     var input models.QuizInput
3     if err := json.NewDecoder(r.Body).Decode(&input); err != nil {
4         http.Error(w, err.Error(), http.StatusBadRequest)
5         return
6     }
7
8     classID, err := primitive.ObjectIDFromHex(input.ClassID)
9     if err != nil {
10         http.Error(w, "Invalid_class_id", http.StatusBadRequest)
11         return
12     }
13
14     quiz := models.Quiz{
15         ID:         primitive.NewObjectID(),
16         Title:      input.Title,
17         ClassID:    classID,
18         Questions: input.Questions,
19         CreatedAt: time.Now(),
20     }
```



```
21
22     collection := helpers.Client.Database("data-feed-db").Collection
        ("quizzes")
23     if _, err := collection.InsertOne(context.Background(), quiz);
        err != nil {
24         http.Error(w, err.Error(), http.StatusInternalServerError)
25         return
26     }
27
28     w.WriteHeader(http.StatusCreated)
29     json.NewEncoder(w).Encode(quiz)
30 }
```

---

Listing 6.3: Funcția CreateQuiz pentru crearea unui quiz nou în MongoDB

---

```
1 r := mux.NewRouter()
2
3 r.Handle("/user/auth/register", handlers.Register).Methods("POST")
4 r.Handle("/user/auth/login", handlers.Login).Methods("POST")
5 r.Handle("/user/me", middleware.JWT(http.Handler(handlers.GetMe)))
6 r.Handle("/user/classes", middleware.JWT(http.Handler(handlers.
    CreateClass))).Methods("POST")
7 // Alte rute...
8
9 corsObj := gorillaHandlers.CORS(
10     gorillaHandlers.AllowedOrigins([]string{"*"}),
11     gorillaHandlers.AllowedMethods([]string{"GET", "POST", "PUT", "
        DELETE", "OPTIONS"}),
12     gorillaHandlers.AllowedHeaders([]string{"Content-Type", "
        Authorization"}),
13 )
14
15 http.ListenAndServe(":8081", corsObj(r))
```

---

Listing 6.4: Definirea rutelor și aplicarea middleware-ului JWT

---

## 6.2 Front-end

Pentru dezvoltarea front-end-ului am folosit React, împreună cu React Router DOM pentru navigare, și tehnologia Three.js prin biblioteca React Three Fiber (R3F) și Drei pentru componente 3D reutilizabile. Această combinație a permis crearea unei interfețe interactive plină de vizualizări 3D dinamice.

### 6.2.1 Configurare front-end

Mediul de dezvoltare a fost configurat cu Node.js și npm, folosind Vite ca bundler și server de dezvoltare rapidă. În directorul proiectului, pachetele relevante instalate includ `react`, `react-router-dom`, `@react-three/fiber`, `@react-three/drei` și alte dependențe utile pentru animații și stilizare. Tehnologiile menționate se îmbină cu folosirea Tailwind CSS pentru stilizare rapidă și eficientă, permițând crearea de interfețe responsive.

### 6.2.2 Dezvoltare front-end

Componenta principală a aplicației este structurată pe baza React și React Router pentru navigare între pagini. Pentru vizualizările 3D s-au folosit componente R3F, cu suport Drei pentru facilități precum `useGLTF` pentru importul modelelor 3D, `useSpring` pentru animații și Canvas pentru spațiul 3D.

Exemple de componente dezvoltate includ:

- **Landing Pages** pentru materii (Astronomie, Chimie, Informatică) cu descrieri și elemente interactive.
- **Formulare** (ex: `ChemistryAddForm`) pentru adăugarea moleculelor cu încărcare fișiere și validări.
- **Vizualizări 3D** (ex: `ChemistryBalloon`, `Drone`, `Island`, `SpaceBackground`) animate și controlate interactiv.
- **Componente educaționale** pentru structuri de date și concepte de programare, cu vizualizări grafice.
- **Dashboard-uri** și pagini pentru quiz-uri, clase, invitații, etc. cu apeluri către API-ul din backend.

Câteva exemple relevante de cod sunt prezentate mai jos:

Un exemplu de componentă standard React care utilizează Three.js pentru a crea un balon animat.

---

```

1  const Balloon = (props) => {
2    const [state, setState] = useState("idle");
3    useEffect(( ) => {
4      const interval = setInterval(( ) => {
5        const states = ["idle", "swayLeft", "swayRight", "tiltForward", "
          tiltBackward", "floatUp", "floatDown"];
6        setState(states[Math.floor(Math.random()*states.length)]);
7      }, 2000);
8      return ( ) => clearInterval(interval);
9    }, []);
10   const { position, rotation } = useSpring({
11     position: state==="floatUp" ? [0,0.3,0] : state==="floatDown" ?
       [0,-0.3,0] : [0,0,0],
12     rotation: state==="swayLeft" ? [0,-0.1,0] : state==="swayRight" ?
       [0,0.1,0] : [0,0,0],
13     config: { duration: 2000 }
14   });
15   return <a.mesh position={position} rotation={rotation} scale={props.scale
       }>...</a.mesh>;
16 };

```

---

Listing 6.5: Exemplu de componentă React pentru un balon animat

Alt exemplu este componenta meniului principal, care include o insulă complexă animată ce poate fi rotită cu mouse-ul.

---

```

1  const Island = ({ isRotating, setIsRotating }) => {
2    const ref = useRef();
3    const lastX = useRef(0);
4    const alfa = 0.0075;
5    const onDown = e => { setIsRotating(true); lastX.current = e.clientX || e.
       touches[0].clientX; };
6    const onMove = e => {
7      if (!isRotating) return;
8      const currentX = e.clientX || e.touches[0].clientX;
9      const delta = (currentX - lastX.current) / window.innerWidth;

```

```
10     lastX.current = currentX;
11     ref.current.rotation.y += delta * alfa * Math.PI;
12 };
13 const onUp = e => setIsRotating(false);
14 useEffect(() => {
15     window.addEventListener("mousedown", onDown);
16     window.addEventListener("mouseup", onUp);
17     window.addEventListener("mousemove", onMove);
18     return () => { ... };
19 }, [isRotating]);
20 return <a.group ref={ref}>...</a.group>;
21 };
```

---

Listing 6.6: Exemplu de animație a insulei meniului principal

### 6.2.3 Conectivitate

Aplicația comunică cu backend-ul prin fetch API, efectuând cereri REST către serverul local pe portul 8000 datorită mediului de dezvoltare în care s-a lucrat. S-au implementat următoarele pattern-uri:

- Obținerea datelor dinamice (ex: formule, molecule, quiz-uri) în componente React folosind `useEffect`.
- Gestionarea token-ului JWT pentru autentificare, salvat în `localStorage` și folosit în header-ul `Authorization`.
- Tratarea erorilor și afișarea mesajelor relevante utilizatorului.
- Formulare pentru inserare, ștergere și actualizare date pe server, cu răspunsuri și validări vizuale în frontend.

Un exemplu de componentă React care face fetch pentru informații pentru planeta Mercur se observă mai jos.

---

```
1 const MercuryFormulas = () => {
2     const [formulas, setFormulas] = useState([]);
3     const [loading, setLoading] = useState(true);
4     const [error, setError] = useState("");
5     useEffect(() => {
```

```
6     fetch("http://localhost:8000/feed/shape/mercury")
7       .then(res => {
8         if(!res.ok) throw new Error();
9         return res.json();
10      })
11      .then(data => setFormulas(Array.isArray(data)?data:[]))
12      .catch(() => setError("Eroare_la_incarcare"))
13      .finally(() => setLoading(false));
14  }, []);
15  return loading ? <p>Loading...</p> : error ? <p>{error}</p> : <ul>{
16    formulas.map(f=><li key={f._id}>{f.formula.name}: {f.formula.expr}</li
      >)}</ul>;
```

---

Listing 6.7: Exemplu de fetch formule chimice cu loading și eroare

Iar o secțiune importantă este vizualizarea 3D a moleculelor, care primește datele de la backend și crează o reprezentare 3D a atomilor și legăturilor chimice folosind Three.js.

---

```
1  const Molecule = ({moleculeId}) => {
2    const [atoms,setAtoms] = useState([]);
3    const [bonds,setBonds] = useState([]);
4    useEffect(() => {
5      fetch(`/feed/chem/molecules/${moleculeId}/3d`)
6        .then(r => r.json())
7        .then(data => { setAtoms(data.atoms); setBonds(data.bonds); });
8    }, [moleculeId]);
9    useEffect(() => {
10     if(groupRef.current) {
11       const box = new THREE.Box3().setFromObject(groupRef.current);
12       const center = new THREE.Vector3();
13       box.getCenter(center);
14       groupRef.current.position.sub(center);
15     }
16   }, [atoms,bonds]);
17   return <group ref={groupRef}>
18     {atoms.map((a,i)=><mesh key={i} position={[a.x,a.y,a.z]}><sphereGeometry
19       args={[0.5,32,32]} /><meshStandardMaterial color="#ccc" /></mesh>)}
20     {bonds.map((b,i)=><mesh key={i} position={[b.x1,b.y1,b.z1]}><lineGeometry
21       args=[2]></mesh></div>
```

---

```

19     {bonds.map((b,i)=> {
20         const start=atoms[b.atom1], end=atoms[b.atom2];
21         if(!start||!end) return null;
22         const startV=new THREE.Vector3(start.x,start.y,start.z);
23         const endV=new THREE.Vector3(end.x,end.y,end.z);
24         const mid = startV.clone().add(endV).multiplyScalar(0.5);
25         const dir = endV.clone().sub(startV).normalize();
26         const len = startV.distanceTo(endV);
27         const quat = new THREE.Quaternion().setFromUnitVectors(new THREE.
                Vector3(0,1,0), dir);
28         return <mesh key={i} position={mid} quaternion={quat}><
                cylinderGeometry args=[0.1,0.1,len,16] /><meshStandardMaterial
                color={0x888888} /></mesh>;
29     })}
30 </group>;
31 };

```

---

Listing 6.8: Vizualizare 3D Molecule (atomi și legături cilindrice)

Iar crearea unei scene complexe ca cea din secțiunea de informatică pentru structura de date coadă cu priorități este realizată mai jos.

---

```

1  class PriorityQueue {
2      constructor(comparator) {
3          this.heap = [];
4          this.compare = comparator;
5      }
6      push(val) { this.heap.push(val); this._siftUp(this.heap.length - 1); }
7      pop() { if (this.heap.length === 0) return null; const top = this.heap[0];
            const last = this.heap.pop(); if (this.heap.length > 0) { this.heap
            [0] = last; this._siftDown(0); } return top; }
8      _shiftUp(idx) { /* mutarea element in sus */ }
9      _shiftDown(idx) { /* mutarea element in jos */ }
10 }
11
12 class TreeNode { constructor(value) { /* crearea unui nod din arbore */ } }
13 const buildTree = (arr) => { /* construirea arborelui dintr-un array */ };
14 const inorder = (node, arr = []) => { /* parcurgerea in ordine */ };

```

```
15 const computePositions = (node, x0, x1, y, gapY, list) => { /* calcularea  
    pozitiilor nodurilor */ };  
16  
17 export default function PriorityQueueDemo() {  
18     const [type, setType] = useState("min");  
19     const [value, setValue] = useState("");  
20     const [pq, setPq] = useState(new PriorityQueue((a, b) => a - b));  
21  
22     const handlePush = () => { if (value === "") return; pq.push(Number(value)  
        ); setValue(""); };  
23     const handlePop = () => { const top = pq.pop(); setPq(pq); };  
24  
25     return (  
26         <div>  
27             <button onClick={handlePush}>push()</button>  
28             <button onClick={handlePop}>pop()</button>  
29             <div>{pq.heap.map((v, i) => <span key={i}>{v}</span>)}</div>  
30         </div>  
31     );  
32 }
```

Listing 6.9: Exemplu de coadă cu priorități 3D folosind React Three Fiber

## Chapter 7

# Scenarii de utilizare

7.1 Înregistrare si autentificare utilizator

7.2 Explorarea meniul principal

7.3 Accesarea secțiunilor educaționale

7.4 Interacțiunea cu scenele 3D educaționale

7.5 Gestionarea profilului de profesor

7.5.1 Crearea de clase și gestionarea elevilor

7.5.2 Crearea de teste și gestionarea lor

7.5.3 Vizualizarea rezultatelor

7.6 Gestionarea contului de elev

7.6.1 Intrarea în clasele profesorului

7.6.2 Accesarea testelor și vizualizarea rezultatelor

7.6.3 Rezolvarea testelor



## Chapter 8

# Evaluarea implementării

### 8.1 Evaluarea back-endului

#### 8.1.1 Testarea back-endului

#### 8.1.2 Monitorizarea back-endului

#### 8.1.3 Evaluarea performanței back-endului

### 8.2 Evaluarea front-endului

#### 8.2.1 Testarea front-endului

#### 8.2.2 Monitorizarea front-endului

#### 8.2.3 Evaluarea performanței front-endului

### 8.3 Testarea infrastructurii / Platformei

## Chapter 9

# Concluzii și perspective

### 9.1 Concluzii

### 9.2 Dezvoltare viitoare

## Appendix A

## Anexe

# Bibliography

- [1] *Three.js*. <https://threejs.org/>.
- [2] *WebGL*. <https://www.khronos.org/webgl/>.
- [3] *React*. <https://reactjs.org/>.
- [4] *Node.js*. <https://nodejs.org/>.
- [5] *MongoDB*. <https://www.mongodb.com/>.
- [6] *Express.js*. <https://expressjs.com/>.
- [7] *Docker*. <https://www.docker.com/>.
- [8] *Kubernetes*. <https://kubernetes.io/>.
- [9] *Git*. <https://git-scm.com/>.
- [10] *GitHub*. <https://github.com/>.
- [11] *Continuous Integration and Continuous Deployment*. <https://www.atlassian.com/continuous-delivery/ci-vs-ci-vs-cd>.
- [12] *Go Language*. <https://golang.org/>.
- [13] M. Horáková, L. Kovářová și P. Doležel, “3D Models and Animations in STEM Education: Czech Experiment,” *\*Central European Journal of Education\**, 2019. <https://link.springer.com/article/10.1007/s10639-024-13210-z>.
- [14] M. Ionescu și A. Popescu, “Distribuția stilurilor de învățare în rândul elevilor din România,” *\*Revista Profesorului\**, 2020. <https://revistaprofesorului.ro/studiu-privind-invatarea-vizuala/>.

- 
- [15] A. Miller, "Learning Styles Among School Students: A Pilot Study," \*British Journal of Educational Psychology\*, 2001. <https://iteach.ro/pagina/1113/>.
- [16] Y. Zhang et al., "VR/AR in STEM Learning," \*PubMed\*, 2024. <https://pubmed.ncbi.nlm.nih.gov/39806348/>.
- [17] Frontiers in Education, "Gamification and Immersive Learning Environments," 2024. <https://www.frontiersin.org/journals/education/articles/10.3389/feduc.2024.1354526/full>.
- [18] Mindomo, "What is Visual Learning?," 2023. <https://www.mindomo.com/blog/what-is-visual-learning/>.
- [19] OECD Education Today, "Can the Targeted Use of Digital Devices Improve Learning?," 2024. <https://oecdutoday.com/can-the-targeted-use-of-digital-devices-in-education-win-over-the-naysayers/>.
- [20] *GLTF Convertor*. <https://gltf.pmnd.rs/>.